

RESEARCH NOTE

Open Access



A deep learning approach: physics-informed neural networks for solving a nonlinear telegraph equation with different boundary conditions

Alemayehu Tamirie Deresse^{1*} and Alemu Senbeta Bekela²

Abstract

The nonlinear Telegraph equation appears in a variety of engineering and science problems. This paper presents a deep learning algorithm termed physics-informed neural networks to resolve a hyperbolic nonlinear telegraph equation with Dirichlet, Neumann, and Periodic boundary conditions. To include physical information about the issue, a multi-objective loss function consisting of the residual of the governing partial differential equation and initial conditions and boundary conditions is defined. Using multiple densely connected neural networks, termed feedforward deep neural networks, the proposed scheme has been trained to minimize the total loss results from the multi-objective loss function. Three computational examples are provided to demonstrate the efficacy and applications of our suggested method. Using a Python software package, we conducted several tests for various model optimizations, activation functions, neural network architectures, and hidden layers to choose the best hyper-parameters representing the problem's physics-informed neural network model with the optimal solution. Furthermore, using graphs and tables, the results of the suggested approach are contrasted with the analytical solution in literature based on various relative error analyses and statistical performance measure analyses. According to the results, the suggested computational method is effective in resolving difficult non-linear physical issues with various boundary conditions.

Keywords Physics-informed neural networks, Non-linear telegraph equation, Boundary conditions

Introduction

Nonlinear PDEs are crucial in many fields of mathematics and physics, including biology, meteorology, geology, optical fibers, plasma physics, and engineering sciences such as quantum mechanics and fluid mechanics [1, 2]. The telegraph equation is a nonlinear hyperbolic PDE.

The transmission line theory equations are used to create telephone equations. They represent the relationships between the currents and voltages on a section of an electric line as a function of the conductor's linear constants (resistance, conductance, inductance, and capacitance). Their resolution enables us to determine how current and voltage change over time at any point along the line [3]. The telegraph equation is commonly used in various fields, including radio frequency, wireless signals, wave propagation, electric communications, cable transmission systems, telephone lines, microwave transmission, and outperforms the heat equation to predict parabolic physical phenomena [4, 5]. The telegraph equation, which is a fundamental non-linear issue, has

*Correspondence:

Alemayehu Tamirie Deresse
alemayehutamirie@mtu.edu.et

¹ Department of Mathematics, Mizan Tepi University, South Western Ethiopia, Tepi 260, Ethiopia

² Department of Mathematics, Samara University, Afar, Samara 132, Ethiopia



© The Author(s) 2025. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

been the focus of extensive computational and analytical research in recent years. For instance, the Taylor series expansion method was used by the writers of the monograph [6] to solve linear and nonlinear fractional telegraph equations. The authors of [7] proposed the bi-finite difference method based on computer algorithms to solve the hyperbolic telegraph equation. The suggested method computer program Mathematica allows for the transformation of the nonlinear problem into a difference equation. The accuracy and conditional stability of the suggested method are discussed and found to be stable and consistent, respectively.

In 2023, Heydari et al. [8] used two collocation techniques based on one- and two-dimensional Romanovski-Jacobi polynomials to address the one- and two-dimensional multi-order time-fractional telegraph problems. Considering numerical instances, the authors verified that the techniques they suggested were accurate. The time-fractional telegraph equations were solved by the authors of [9] by using a mixture of analytical methods, the Laplace transform operator, and the rational power series techniques. The derivative of the fractional order is expressed using the Caputo operator. In the work of [10], the author implemented a conformable Laplace transform methodology mixed with the new iterative technique to find the analytical solution of the time-fractional telegraph equation in two dimensions. The Laplace transform is used to solve the linear portion of the problem under consideration. In contrast, the iterative method's sequential iteration approach eliminates the noise terms in the nonlinear part, and a single iteration yields an exact answer. The authors of [11] put out a novel approach for approximating the solutions of hyperbolic telegraph equations that arise in magnetic fields and electrical impulse transmissions. They invented the Laplace-Carson homotopy perturbation approach, which combines the homotopy perturbation technique with the Laplace-Carson transform. Monographs [12–15] provide extra information.

In order to solve a nonlinear time-fractional telegraph equation, this paper aims to introduce physics-informed neural networks (PINNs), an algorithm based on machine learning. To push the network toward a solution to the given problem, PINNs employ optimization techniques to iteratively adjust the parameters of a neural network till the magnitude of a defined, physics-informed function of loss is reduced to an acceptable level [16, 17]. It contains terms that represent the beginning and boundary conditions along the time and space domain boundary, together with the PDE residual at particular domain points (referred to as collocation) [18]. An information processing system operating in parallel, certain characteristics of ANNs are similar to those of particular

brain activities. It can perform complex computations and is composed of neurons and synaptic weights [19]. The network mimics the work of the biological human brain [20]. A biological neuron receives input from other sources, combines them in some way, performs a general non-linear operation on the result, and then outputs the final result [21, 22]. The contributors of [23] describe a technique that creates an ankle joint gait trajectory from a six-bar linkage mechanism of fixed dimensions by combining a deep learning approach with a genetic algorithm. They used the generated data to train long short-term memory models and a feedforward neural network by simulating the kinematic behavior of the six-bar linking mechanism within specified mechanical limitations. In the study [24], the authors optimize the weights and parameters of a neural network model to forecast accurate and dependable solutions for the jamming transition in traffic flow using the supervised ANNs. The writers of [25] used controlled learning procedures of machine learning algorithms based on ANNs to explore the solution of nonlinear ordinary differential equations (ODEs) involving nanofluid flow in rotating systems. The system equations have no closed-form analytical solution. They found that the method can also produce accurate results for non-linear systems that have no analytical solution. ANN is attractive for approximating extremely nonlinear processes. However, because solving an algebraic equation is usually simpler than solving highly nonlinear, large-scale optimization problems related to neural network training, PDE solvers built on ANNs are generally unable to compete with classical numerical solution methods, especially in low to moderate sizes. In addition, they do not yet have the sophisticated error analysis that has been developed for conventional numerical approaches. For this reason, numerous specialized techniques have been created over time for particular issues, frequently embedding restrictions or incorporating fundamental physical assumptions into the predictions. The inverse multiquadric (IMQ) radial basis function (RBF) is used as an activation function by the authors of [26, 27] to study the wire coating problem and magnetohydrodynamics (MHD) Casson nanofluid flow in a porous medium along a stretchy surface with various slip conditions and Oldroyd 8-constant fluid based on ANNs. In their novel approaches to learning, genetic algorithms, and sequential quadratic programming are hybridized. Consequently, Butt et al. [28] use inverse multiquadric (IMQ) radial basis neural networks (RBNNs) as a new method to study the effects of magnetohydrodynamics on two-dimensional nanofluid boundary layer flow under the influence of radiation in a porous media. By using feed-forward neural networks trained with a hybrid composition of genetic algorithms (GAs) and sequential

quadratic programming (SQP), study [29] offers the use of a well-known numerical-based neuro-evolution heuristic technique.

One such technique is the PINNs, which can be applied to almost any DE and is appealing for rapid prototyping where effectiveness and great accuracy are not the primary objectives [30, 31].

Raissi et al. [32] presented promising results indicating that, considering a significant number of collocation points, “PINNs can achieve excellent accuracy of predictions if the provided PDE is well posed.” PINNs search to find neural networks that minimize loss among an assortment of neural networks. A minimizer provides a close approximation of the PDE’s solution. PINNs have altered the traditional variational notion, which minimizes an energy function. The formulation of PINNs can be used for wide classes of PDEs and does not require that the examined PDE have a variational principle; this is the primary difference between the two. The authors of [33] presented PINNs as a deep learning framework for PDE problems of both data-driven PDE discovery and data-driven solutions. They used the Schrödinger equation as an example, applying the continuous-time and discrete-time formulations of their respective algorithms. Blechschmidt and Ernst [34] used PINNs on the backward stochastic DEs to solve higher-dimensional nonlinear PDEs using neural networks. According to the authors, the approach provides appealing approximation capabilities for high-dimension and strongly nonlinear problems. On top of that, Schäfer [35] compares ANNs and PINNs and concludes that ANNs are less accurate than PINNs. Recently, the authors of [36] used PINNs for solving two-dimensional hyperbolic non-linear sine Gordon equations with different boundary conditions. They found that the method outperformed the other numerical methods in terms of accuracy and reliability. The authors also provide the theoretical error bounds of PINNs to confirm the effectiveness of the method. The additional information can be found in the references [37–42].

Numerous techniques have been devised to solve the telegraph equation, as we attempted to highlight in the literature mentioned above. Nevertheless, high-dimensional, inverse and highly non-linear problems cannot be efficiently approximated using these approaches. As a result, researchers cautiously turn to machine learning-based techniques like ANNs to find solutions for such problems. To mention some machine learning methods utilized for solving the telegraph equation, the authors of the paper [43] used ANNs for solving the one-dimensional time fractional linear telegraph equation, but they did not work for non-linear and different boundary conditions. Jacobi DNN was utilized by the authors of the work [44] to solve the one- and two-dimensional linear

integer-order telegraph equation. However, the non-linear telegraph equation was not taken into account by the writers. Also, both studies [43] and [44] does not include training for various neural network designs, activation functions, model optimizations, or related hyperparameter tuning. In this paper, we used the modified and specialized version of ANNs known as PINNs to solve the non-linear space-time telegraph equation with different boundary conditions. PINNs is a method of deep learning that integrates physical rules or restrictions into the process of learning [45]. They aim to leverage both data-driven learning and prior knowledge of the underlying physics governing a system [46–48]. Additionally, because PINNs is mesh-free, it offers the advantage of avoiding mesh-related problems. High-dimensional linear and non-linear issues, problems involving uncertainty, parameter estimation, and inverse problems are all effectively handled by PINNs. To the best of the author’s knowledge, the PINNs have never been used to solve the suggested non-linear telegraph equation in the body of existing research, and hence, the current study is fresh and unique. The following is the nonlinear telegraph equation utilized in this work:

$$\frac{\partial^2 u(\mathbf{x}, t)}{\partial t^2} + \beta \frac{\partial u(\mathbf{x}, t)}{\partial t} + \alpha N(u(\mathbf{x}, t)) = c \Delta(u(\mathbf{x}, t)) + f(\mathbf{x}, t), \quad (1)$$

where $\mathbf{x} = (x_1, x_2, \dots, x_d) \in \Omega \in \mathbb{R}^d$ and Δ is the Laplacian operator. $f(\mathbf{x}, t)$ is the source term, N is a nonlinear operator, which means $N(u(\mathbf{x}, t))$ denotes the nonlinear term in (1), while α , β , and c are referred to as real constants.

When $d = 1$, the problem (1) reduced to (1+1)-dimensional telegraph equation, with the initial conditions

$$u(x, 0) = f_1(x), \quad u_t(x, 0) = f_2(x), \quad x \in [a, b] \quad (2)$$

and Dirichlet or Periodic or Neumann boundary conditions

$$u(a, t) = k_1(t), \quad u(b, t) = k_2(t), \quad \text{or} \quad (3)$$

$$u(a, t) = u(b, t), \quad u_x(a, t) = u_x(b, t) \quad \text{or} \quad (4)$$

$$u_x(a, t) = g_1(t), \quad u_x(b, t) = g_2(t). \quad (5)$$

This is how the remainder part of this article is structured: The essential preliminary facts of PINN are covered in Section 2. The general procedure of the claimed approach, PINNs for solving the non-linear telegraph equation, is presented in Section 3. Numerical examples for Dirichlet, Periodic, and Neumann boundary

conditions are used in Section 4 to validate the suggested technique. Moreover, several neural network experiments are conducted to choose the best neural network architecture, activation function, hidden layers, number of collocation points, learning rate, and model optimization for the given issue. Subsequently, closing thoughts and perspectives are drawn in Section 5.

Basic preliminaries of PINNs

This section will go over some crucial concepts and aspects of PINNs.

Artificial Neural Networks (ANNs)

Many deep learning strategies are sometimes incapable of handling some application cases because they are too complex or too broad. However, ANNs, as they are popularly known, represent deep learning using artificial intelligence [49]. Fig. 1 illustrates the ANN design of a single layer with three nodes that can be used to approximate a solution to a three-dimensional problem.

Definition 2.1 (Mathematical definition of ANN [50, 51]) Let $d \in \mathbb{N}$. We define an artificial neuron $u : \mathbb{R}^d \rightarrow \mathbb{R}$ as a mapping with weight $\mathbf{w} \in \mathbb{R}^d$, bias $b \in \mathbb{R}$, and activation function $\sigma : \mathbb{R} \rightarrow \mathbb{R}$. The neuron's output is given by the expression

$$\begin{aligned} u &= \sigma \left(\sum_{i=1}^d w_i x_i + b \right) = \sigma(\mathbf{w}\mathbf{x}^T + b) \\ &= \sigma(\mathbf{w} \cdot \mathbf{x} + b), \text{ for } \mathbf{x} \in \mathbb{R}^d. \end{aligned} \quad (6)$$

Equation (6) denotes an ANN of a single layer. For architecture having more than one hidden layer, the output in Definition 2.1 is used as an input for the next layer prediction. According to the forward propagation of data from the input to the output layers explained in [52, 53], if we denote $\mathbf{z}^{(1)} = \mathbf{w}^{(1)} \cdot \mathbf{x}^{(1)} + b^{(1)}$, then the network output is given by $\mathbf{a}^{(1)} = \sigma^{(1)}(\mathbf{z}^{(1)})$. That means, at the first layer, we have

$$\begin{aligned} \mathbf{z}^{(1)} &= \mathbf{w}^{(1)} \cdot \mathbf{x}^{(1)} + b^{(1)}, \\ \mathbf{a}^{(1)} &= \sigma^{(1)}(\mathbf{z}^{(1)}). \end{aligned} \quad (7)$$

Again, the output $\mathbf{a}^{(1)}$ serves as the input for the neurons of the next layer and at the second layer, we have

$$\begin{aligned} \mathbf{z}^{(2)} &= \mathbf{w}^{(2)} \cdot \mathbf{a}^{(1)} + b^{(2)}, \\ \mathbf{a}^{(2)} &= \sigma^{(2)}(\mathbf{z}^{(2)}). \end{aligned} \quad (8)$$

Then, the network training is continued until the desired prediction is obtained, and the prediction for the neuron in the ℓ^{th} layer is calculated by the formula

$$\begin{aligned} \mathbf{a}^{(\ell)} &= \sigma^{(\ell)}(\mathbf{z}^{(\ell)}) = \sigma^{(\ell)}(\mathbf{w}^{(\ell)} \cdot \mathbf{a}^{(\ell-1)} + b^{(\ell)}), \\ \ell &= 1, \dots, L-1, \end{aligned} \quad (9)$$

where $\mathbf{a}^{(\ell-1)}$ is the output of the previous layer $\ell-1$.

Finally, the output at the L^{th} layer is calculated by the formula

$$\mathbf{a}^{(L)} = \mathbf{w}^{(L)} \cdot \mathbf{a}^{(L-1)} + b^{(L)}. \quad (10)$$

ANN typically avoids applying the activation function to the output. However, the particular problem being solved determines whether or not an activation function is applied to the output. Using an activation function could limit the output range if the model is required to generate

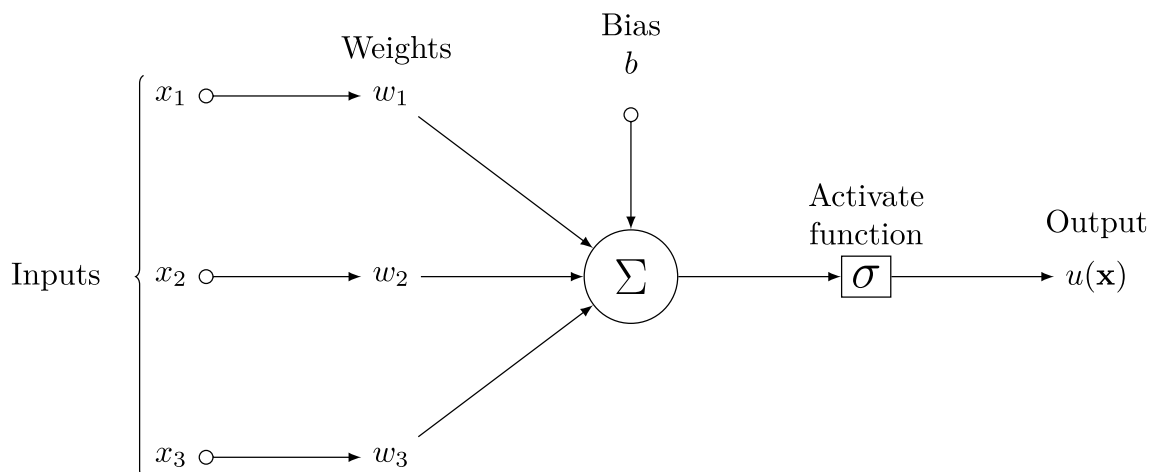


Fig. 1 Schematic illustration of a single-layer ANN comprising one neuron in the output layer and three neuron in the input layer

continuous, real-valued outputs. However, an activation function is required for jobs that require constraints, such as classification or bounded output.

Activation function

An activation function σ determines the output of a neuron by applying a mathematical transformation to its weighted input sum, introducing non-linearity and enabling the network to capture complex patterns in the data [54]. In other words, when a node (or layer) receives an array of input values, the activation function must generate the desired outcome [55]. The most widely used activation functions for PINN are [56]: sigmoid or logistic function ($\sigma(x) = 1/(1 + \exp(-x))$), sinusoidal function ($\sin$), Swish (an extension of the sigmoid-weighted linear unit function (SiLU)), hyperbolic tangent function ($\sigma(x) = \tanh(x)$), and ReLU (positive linear function ($\sigma(x) = \max(0, x)$)). In this paper, we perform neural network training for various activation functions to select which activation function produces the best results.

Deep feed-forward neural networks

With its potent tools for pattern recognition, classification, and predictive modeling, ANN has completely transformed the field of machine learning. Of all the different types of neural networks, the feedforward neural network (FNN) is among the most basic and popular [57]. FNN is a kind of ANN in which node-to-node connections don't create cycles [58]. This feature sets it apart from the recurrent neural network (RNN). The network consists of an input layer, one or more hidden layers, and an output layer. Information flows in one direction—from input to output—hence the name “feedforward” [59]. To create PINNs, three ANN architectures are frequently employed: FNN, convolutional neural network, and long-short-term memory neural network [60].

Definition 2.2 (Deep FNN [61, 62]) A deep FNN is a function of the form

$$\hat{Y} := F_{W,b}(X) = \left(f_{W^{(L)},b^{(L)}}^{(L)} \circ \dots \circ f_{W^{(1)},b^{(1)}}^{(1)} \right)(X),$$

where $f_{W^{(\ell)},b^{(\ell)}}^{(\ell)} := \sigma^{(\ell)}(W^{(\ell)}X + b^{(\ell)})$, $\forall \ell \in \{1, \dots, L\}$ is a semi-affine function, where $\sigma^{(\ell)}$ is a univariate and continuous non-linear activation function such as $\tanh(\cdot)$ and $\max(\cdot, 0)$. $W = (W^{(1)}, \dots, W^{(L)})$ and $b = (b^{(1)}, \dots, b^{(L)})$ are weight matrices and bias vectors, respectively.

Definition 2.3 (Semi-Affine Functions [61]) Let $\sigma : \mathbb{R} \rightarrow B \subset \mathbb{R}$ denote a continuous, monotonically

increasing function whose codomain is a bounded subset of the real line. A function $f_{W^{(\ell)},b^{(\ell)}}^{(\ell)} : \mathbb{R}^n \rightarrow \mathbb{R}^m$, given by

$$f(u) = W^{(\ell)} \cdot \sigma^{(\ell-1)}(u) + b^{(\ell)}, \quad W^{(\ell)} \in \mathbb{R}^{m \times n}, \quad b^{(\ell)} \in \mathbb{R}^m,$$

is a semi-affine function in u , e.g. $f(u) = w \tanh(u) + b$. $\sigma(\cdot)$ are the activation functions of the output of the previous layer.

Definition 2.4 The following procedures are carried out in the machine learning language to solve any PDE with PINNs [45].

1. Design an ANN $\hat{u}(x, t; \theta)$ as a surrogate of the true solution $u(x, t)$.
2. Build a training set that is used to train the neural network.
3. Define an appropriate loss function that accounts for residuals of the PDE, initial, boundary, and final conditions, and
4. Train the network by minimizing the cost function defined in the previous Step 3.

Method

In this section, based on the PINNs procedure given in Definition 2.4, we create the PINNs strategies to approximate the outcome $u : [0, T] \times \Omega \rightarrow \mathbb{R}$ of tackling the issue (1) in one dimension. Considering the non-linear telegraph equation as follows:

$$\begin{aligned} \frac{\partial^2 u(x, t)}{\partial t^2} + \beta \frac{\partial u(x, t)}{\partial t} + \alpha N(u(x, t)) \\ = c \frac{\partial^2 u(x, t)}{\partial x^2} + f(x, t), \end{aligned} \quad (11)$$

with the initial conditions

$$u(x, 0) = f_1(x), \quad x \in \Omega, \quad (12a)$$

$$u_t(x, 0) = f_2(x), \quad x \in \Omega, \quad (12b)$$

and Dirichlet boundary conditions,

$$u(a, t) = k_1(t), \quad t \in [0, T], \quad (13a)$$

$$u(b, t) = k_2(t), \quad t \in [0, T], \quad (13b)$$

where $(x, t) \in (0, T] \times \Omega$, $\Omega = \{x : a \leq x \leq b\} \subset \mathbb{R}^d$ is a domain that has boundaries, and T represents the final time. Let $N(u(x, t))$ be a nonlinear function of $u(x, t)$. For instance, it could involve terms like $u^n(x, t)$, $e^{u(x, t)}$, or any other nonlinear transformation. We assumed this

operator is well-defined and differentiable, as PINNs rely on automatic differentiation (AD). To solve the telegraph equation (11) with conditions (12) and (13) through the proposed technique, we employed the following steps adapted from Definition 2.4 and the PINNs algorithm 1 of reference [45].

Step 1. Neural Network: We chose the neural network architecture having two neurons $(x, t) = (x_1, x_2)$ in the input layer and one neuron $u(x, t) = u(x_1, x_2)$ in the output layer. Regarding the choice of hidden layers, we considered four layers along with 50 neurons per hidden layer. The neural network structure is fed into the training algorithm with the help of deep FNN, sometimes referred to as multilayer perceptron (MLP). The derivative(s) of any order of the network via any trainable parameter is computed based on automatic machine learning differentiation [63].

Step 2. Training Dataset: The general training set $\mathcal{T}$ of the PINNs model is selected in the interior domain $\mathcal{T}_{int} \subset (a, b) \times (0, T)$ and on the boundaries $\mathcal{T}_{x=a} \subset \{a\} \times [0, T]$, $\mathcal{T}_{x=b} \subset \{b\} \times [0, T]$, $\mathcal{T}_{t=0} \subset (a, b) \times \{0\}$. Thus

$$\mathcal{T} = \mathcal{T}_{int} \cup \mathcal{T}_{x=a} \cup \mathcal{T}_{x=b} \cup \mathcal{T}_{t=0}.$$

Step 3. Loss function: The PINNs method constructs a neural network approximation $\hat{u}(x, t; \theta) \approx u(x, t)$ of the solution of non-linear telegraph equation (11), where $\hat{u} : [0, T] \times \Omega \rightarrow \mathbb{R}$ denotes a function realized by a neural network with parameter θ . The PDE residual of a given neural network approximation $\hat{u} : [0, T] \times \Omega \rightarrow \mathbb{R}$ of the solution u , is given by

$$R_{pde}(x, t; \theta) := \frac{\partial^2 \hat{u}(x, t)}{\partial t^2} + \beta \frac{\partial \hat{u}(x, t)}{\partial t} + \alpha N(\hat{u}(x, t)) - c \frac{\partial^2 \hat{u}(x, t)}{\partial x^2} - f(x, t). \quad (14)$$

The residuals of initial conditions in (12) are respectively given by

$$R_{ic1}(x, 0; \theta) := \hat{u}(x_i, 0) - k_1(t_i). \quad (15a)$$

$$R_{ic2}(x, 0; \theta) := \hat{u}_t(x_i, 0) - k_2(t_i). \quad (15b)$$

The residuals of boundary conditions in (13) are respectively given by

$$R_{bc1}(a, t; \theta) := \hat{u}(a, t_i) - f_1(x_i), \quad (16a)$$

$$R_{bc2}(b, t; \theta) := \hat{u}_t(b, t_i) - f_2(x_i). \quad (16b)$$

The loss function corresponding to the PDE residual R_{pde} (14) is

$$L_{pde}(\theta; \mathcal{T}_{int}) = \frac{1}{|\mathcal{T}_{int}|} \sum_{(x_i, t_i) \in \mathcal{T}_{int}} |r(x_i, t_i; \theta)|^2. \quad (17)$$

The loss functions corresponding to the initial conditions residuals in Eq. (15) are respectively given by

$$L_{ic1}(\theta; \mathcal{T}_{t=0}) = \frac{1}{|\mathcal{T}_{t=0}|} \sum_{(x_i, t_i) \in \mathcal{T}_{t=0}} |\hat{u}(x_i, 0) - f_1(x_i)|^2, \quad (18a)$$

$$L_{ic2}(\theta; \mathcal{T}_{t=0}) = \frac{1}{|\mathcal{T}_{t=0}|} \sum_{(x_i, t_i) \in \mathcal{T}_{t=0}} |\hat{u}_t(x_i, 0) - f_2(x_i)|^2. \quad (18b)$$

The loss functions corresponding to the boundary conditions residuals presented in Eq. (16) are respectively given by

$$L_{bc1}(\theta; \mathcal{T}_{x=a}) = \frac{1}{|\mathcal{T}_{x=a}|} \sum_{(x_i, t_i) \in \mathcal{T}_{x=a}} |\hat{u}(a, t_i) - k_1(t_i)|^2, \quad (19a)$$

$$L_{bc2}(\theta; \mathcal{T}_{x=b}) = \frac{1}{|\mathcal{T}_{x=b}|} \sum_{(x_i, t_i) \in \mathcal{T}_{x=b}} |\hat{u}(b, t_i) - k_2(t_i)|^2. \quad (19b)$$

To integrate all conditions into the PINNs loss function, we used a multi-objective loss function consisting of the loss functions corresponding to the residual of the governing PDE, initial conditions, and boundary conditions. Hence, for solving the given initial value problem, the loss functions (17), (18), and (19) are all summed together and considered as a single total loss function

$$L_{total}(\theta; \mathcal{T}) = L_{pde}(\theta; \mathcal{T}_{int}) + L_{ic1}(\theta; \mathcal{T}_{t=0}) + L_{ic2}(\theta; \mathcal{T}_{t=0}) + L_{bc1}(\theta; \mathcal{T}_{x=a}) + L_{bc2}(\theta; \mathcal{T}_{x=b}). \quad (20)$$

Neumann boundary conditions case

For Neumann conditions, the Dirichlet conditions in (13) become

$$u_x(a, t) = g_1(t), \quad (21a)$$

$$u_x(b, t) = g_2(t). \quad (21b)$$

The residuals of Neumann conditions in (21) are respectively given by

$$R_{bc3}(a, t; \theta) := \hat{u}_x(a, t_i) - g_1(x_i), \quad (22a)$$

$$R_{bc4}(b, t; \theta) := \hat{u}_x(b, t_i) - g_2(x_i). \quad (22b)$$

The loss functions corresponding to the Neumann conditions residuals presented in Eq. (22) are respectively given by

$$L_{bc3}(\theta; T_{x=a}) = \frac{1}{|T_{x=a}|} \sum_{(x_i, t_i) \in T_{x=a}} |\hat{u}_x(a, t_i) - g_1(t_i)|^2, \quad (23a)$$

$$L_{bc4}(\theta; T_{x=b}) = \frac{1}{|T_{x=b}|} \sum_{(x_i, t_i) \in T_{x=b}} |\hat{u}_x(b, t_i) - g_2(t_i)|^2. \quad (23b)$$

Hence, the total loss function (20) for the Neumann boundary value problem becomes

$$L_{\text{total}}(\theta; T) = L_{\text{pde}}(\theta; T_{\text{int}}) + L_{\text{ic1}}(\theta; T_{t=0}) + L_{\text{ic2}}(\theta; T_{t=0}) + L_{bc3}(\theta; T_{x=a}) + L_{bc4}(\theta; T_{x=b}). \quad (24)$$

Periodic boundary conditions case

Periodic boundary conditions are an essential concept in computational modeling, especially in simulations of physical systems. They are used for the modeling of an infinite system by taking into account a finite representative volume, where the boundaries of the system are connected such that the behavior on one edge of the domain seamlessly continues onto the opposite edge. This method is frequently applied in domains such as materials science, molecular dynamics, and computational fluid dynamics. When using deep neural networks to solve PDEs, the authors of the paper [64] describe how to represent periodic functions and precisely enforce periodic boundary conditions into the training loss. According to [64], for periodic boundary conditions, the Dirichlet conditions in (13) become as follows:

$$u(a, t) = u(b, t), \quad (25a)$$

$$u_x(a, t) = u_x(b, t). \quad (25b)$$

The residuals of periodic conditions in (25) are respectively given by

$$R_{bc5}(a, b, t; \theta) := \hat{u}(a, t_i) - \hat{u}(b, t_i), \quad (26a)$$

$$R_{bc6}(a, b, t; \theta) := \hat{u}_x(a, t_i) - \hat{u}_x(b, t_i). \quad (26b)$$

The loss functions corresponding to the periodic conditions residuals in Eq. (26) are respectively given by

$$L_{bc5}(\theta; T_{x=a,b}) = \frac{1}{|T_{x=a,b}|} \sum_{(x_i, t_i) \in T_{x=a,b}} |\hat{u}(a, t_i) - \hat{u}(b, t_i)|^2, \quad (27a)$$

$$L_{bc6}(\theta; T_{x=a,b}) = \frac{1}{|T_{x=a,b}|} \sum_{(x_i, t_i) \in T_{x=a,b}} |\hat{u}_x(a, t_i) - \hat{u}_x(b, t_i)|^2. \quad (27b)$$

Hence, the total loss function (20) for periodic boundary value problem becomes

$$L_{\text{total}}(\theta; T) = L_{\text{pde}}(\theta; T_{\text{int}}) + L_{\text{ic1}}(\theta; T_{t=0}) + L_{\text{ic2}}(\theta; T_{t=0}) + L_{bc5}(\theta; T_{x=a}) + L_{bc6}(\theta; T_{x=b}). \quad (28)$$

Step 4. Training Process: Using the training samples (which are components of the domain and boundaries), we leverage the loss function (20) for Dirichlet, (24) for Neumann, and (28) for periodic boundary conditions problem as the final stage in the PINNs algorithmic framework. The schematic diagram in Fig. 2 illustrates how PINNs is trained to solve (11).

To train our PINNs model, we used the Glorot normal weight initialization with the learning rate 0.001. Taking the non-linear telegraph equation with different boundary conditions, we performed various experiments on the suggested neural network structure on the selection of the best activation function, optimization algorithm, and neural network architecture used to get the optimal solution to the proposed problem. Throughout different neural network training, we assessed the accuracy of the model by comparing the precise solution $u(x^i, t^i)$ with the predicted value $\hat{u}(x^i, t^i)$ obtained by the suggested model for each illustrative instance. The comparison was made using L_2 , L_∞ , and RMSE defined as follows,

$$\text{RMSE} = \sqrt{\frac{\sum_{i=1}^T |u(x^i, t^i) - \hat{u}(x^i, t^i)|^2}{T}}, \quad (29)$$

$$L_\infty = \max_{1 \leq i \leq T} |u(x^i, t^i) - \hat{u}(x^i, t^i)|^2, \quad (30)$$

$$L_2\text{-error} = \frac{\sqrt{\sum_{i=1}^T |u(x^i, t^i) - \hat{u}(x^i, t^i)|^2}}{\sqrt{\sum_{i=1}^T \hat{u}(x^i, t^i)^2}}, \quad (31)$$

where T is the general training data set of the model. To further confirm the application and performance of PINNs, we give various statistical performance measures, including mean absolute error (MAE), mean squared error (MSE), range error, variance error, and standard deviation error, in addition to error analysis (29), (30), and (31).

Numerical results and discussion

In this section, we used a Python software package shown in the paper [65] as a computational tool and employed the PINNs algorithm to examine the applicability and soundness of the suggested model. As an illustrative example, we considered three types of boundary conditions: Dirichlet, Periodic, and Neumann conditions.

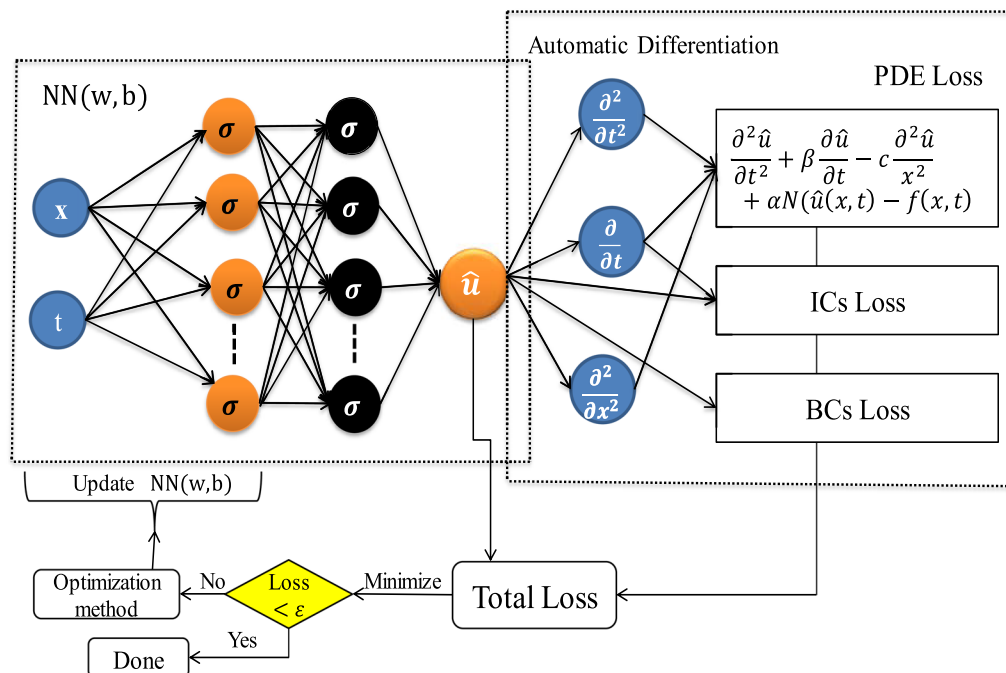


Fig. 2 Schematic diagram of a PINNs for solving the non-linear telegraph equation (11)

Example 4.1 Nonlinear telegraph equation with Dirichlet BCs

In the first test, we considered the following nonlinear telegraph equation,

$$\frac{\partial^2 u}{\partial t^2} + 2 \frac{\partial u}{\partial t} + u^3 = \frac{\partial^2 u}{\partial x^2} - u, \quad -2 \leq x \leq 2, \quad 0 \leq t \leq 5, \quad (32)$$

with the initial conditions

$$u(x, 0) = \frac{1}{2} + \frac{1}{2} \tanh\left(\frac{x}{8} + 5\right), \quad -2 \leq x \leq 2, \quad (33a)$$

$$u_t(x, 0) = \frac{3}{16} + \frac{3}{16} \tanh\left(\frac{x}{8} + 5\right)^2, \quad -2 \leq x \leq 2, \quad (33b)$$

and boundary conditions

$$u(-2, t) = \frac{1}{2} + \frac{1}{2} \tanh\left(-\frac{1}{4} + \frac{3t}{8} + 5\right), \quad 0 \leq t \leq 5, \quad (34a)$$

$$u(2, t) = \frac{1}{2} + \frac{1}{2} \tanh\left(\frac{1}{4} + \frac{3t}{8} + 5\right), \quad 0 \leq t \leq 5. \quad (34b)$$

The reference solution to BVP (32)–(34) is given by $u(x, t) = \frac{1}{2} + \frac{1}{2} \tanh\left(\frac{x}{8} + \frac{3t}{8} + 5\right)$ [66].

Using PINNs, we train the proposed neural network architecture to minimize the loss function (20) based on steps outlined in Section 3 to solve problem (32) with conditions (33a)–(34b). To accomplish this, first, we performed various training to choose the optimal hyperparameters, including activation function, neural network design, hidden layers, and model optimization.

Discussion on the selection of optimization algorithm For PINNs to successfully train and identify a solution that meets the data and the governing physical principles, the optimization algorithm selection is essential. PINNs involve complex loss functions that mix data-based, physics-based, and perhaps other, making the optimization process challenging. To choose the optimal optimization technique for the given issue, we experimented with five well-known optimization algorithms, including stochastic gradient descent (SGD), root mean squared propagation (RMSProp), Adam, L-BFGS-B, and their combination. As we can see from Table 1 and Fig. 3 the L-BFGS-B outcome is superior to Adam, SGD, and RMSProp. The mixture of Adam and L-BFGS-B optimizations outperforms the other optimizers. As seen in Fig. 3d, LBFGS-B shows more consistent convergence with few iterations in contrast to other optimizations. Compared to SGD and RMSProp, the Adam optimizer gives minimum errors. As we can observe from Figs. 3a and 3b the optimizers SGD and RMSProp do not perform well for the suggested

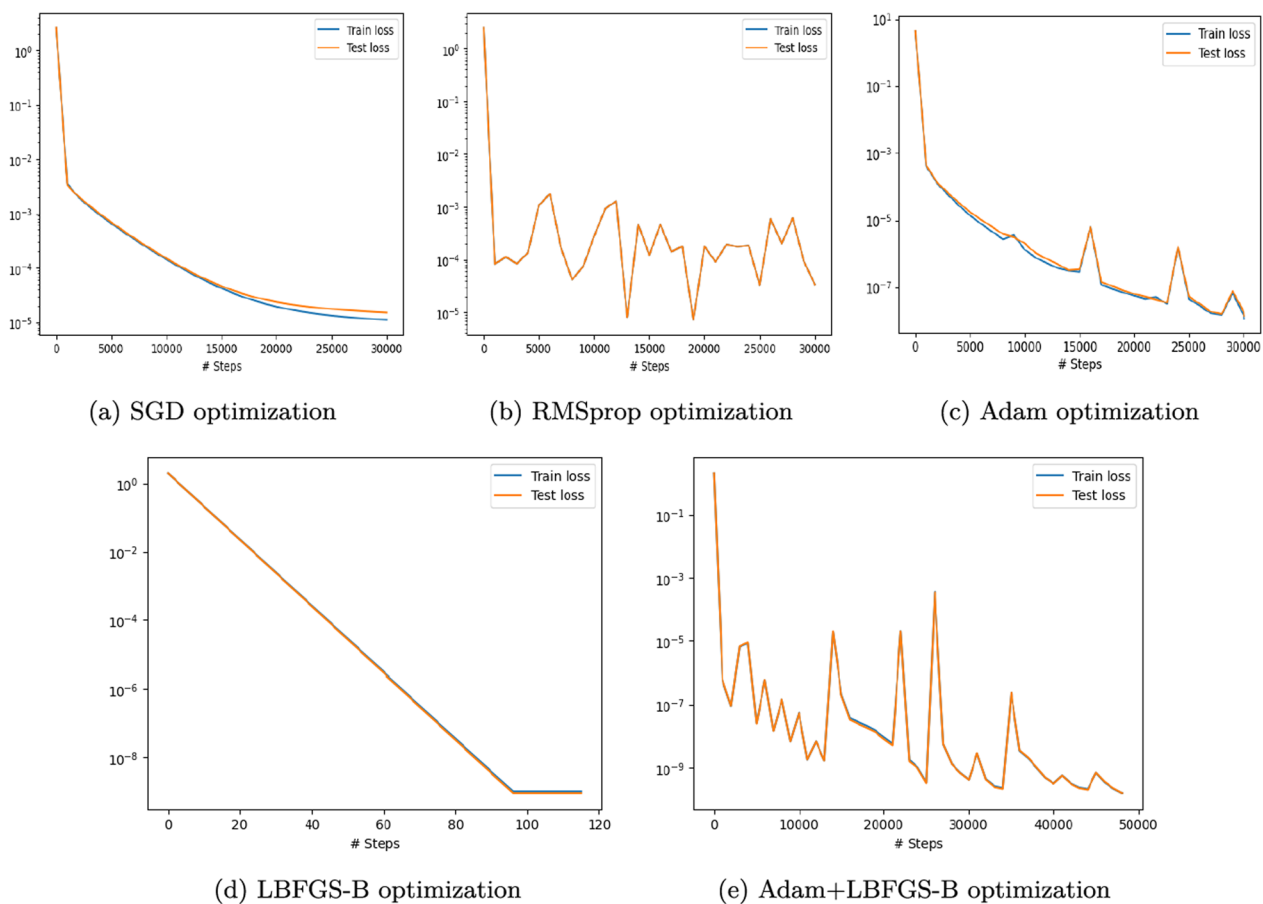


Fig. 3 Train and test loss to the problem (32) using the optimization algorithms **a** SGD, **b** RMSprop, **c** Adam, **d** LBFGS-B, and **e** LBFGS-B+Adam

problem. However, the prediction obtained via SGD is smooth and quickly converges more rapidly than the one obtained based on RMSProp. By employing a single or small batch of training samples at each stage, SGD computes approximate gradients, making it scalable for large datasets [67].

The array of steps (epochs) in Fig. 3 demonstrates how many iterations are employed to train the neural networks of the model, which implies how many times the network weights are updated. We set the model's epoch count to 30000 in this paper. Figs. 3c, d, and e show the train as well as the test losses are decreasing when the training iteration is increasing. This implies that by refining its estimate, the model is moving closer to resolving the issue at hand. In Fig. 3c, we trained the neural networks using the Adam optimization for 30000 epochs. Since the neural networks are trained until convergence and the L-BFGS-B optimizer does not need the learning rate, the number of iterations is likewise disregarded in L-BFGS-B. Moreover, LBFGS-B is a second-order optimization algorithm, which means it uses the Hessian matrix (or an approximation of it)

to update the parameters. This makes LBFGS-B computationally more expensive than first-order algorithms like Adam, which only use the gradient information. Hence, to keep its computational feasibility, LBFGS-B requires smaller batch sizes. For such a reason, training the PINNs model with LBFGS-B in Fig. 3d results in a lesser loss with fewer iterations than training with Adam optimization. After training with Adam optimization, we continued the training using L-BFGS-B, and we got the minimum train and test loss compared to the loss obtained with other optimizations as indicated in Fig. 3e.

Discussion on the selection of activation function According to [68, 69] the activation function selection depends on the specific problems under consideration. Using mixed Adam and L-BFGS-B optimization, we performed a comparison of a few popular activation functions to ascertain which activation most effectively decreased the loss function of our PINNs model. To ascertain which function is suitable, we made a comparison among some popular activation functions. As reported in Table 2 and Fig. 4b, through the application

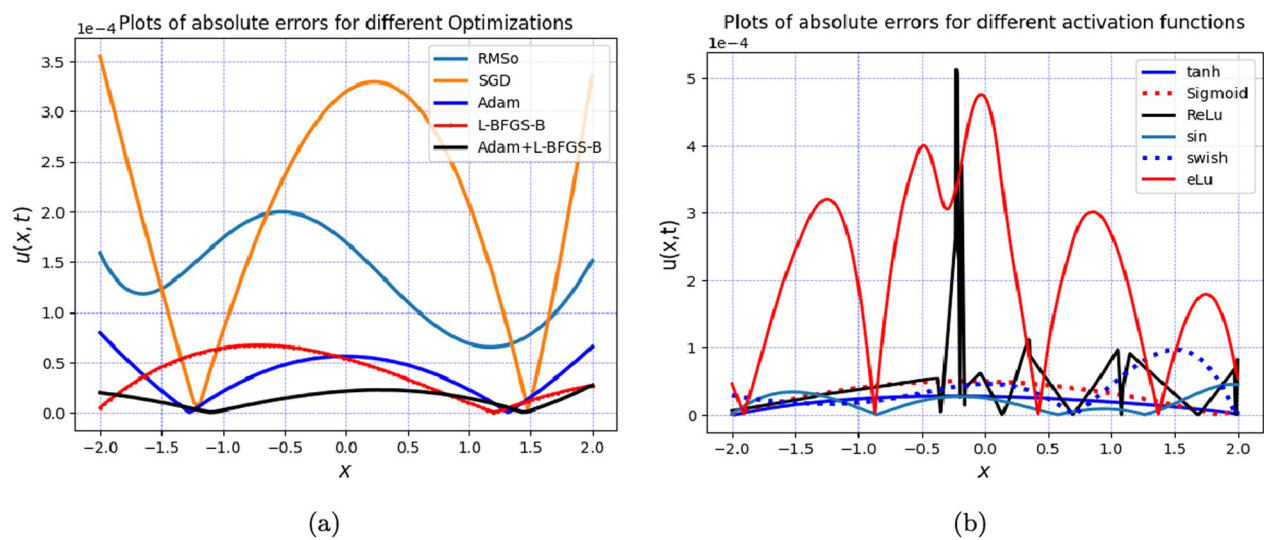


Fig. 4 The line plots of absolute errors for different **a** model optimizations at $t = 0.5$ and **b** activation functions at $t = 0.5$

Table 1 PINNs approximation errors, train and test losses, and model training time for optimization algorithms

Optimization	L_2	L_∞	RMSE	Train loss	Test loss
SGD	1.622e-04	2.288e-04	1.622e-05	8.50e-06	7.36e-06
RMSProp	6.047e-04	5.296e-03	5.730e-05	6.57e-06	6.71e-07
Adam	7.802e-06	2.047e-05	7.139e-06	2.15e-09	3.12e-09
L-BFGS-B	5.363e-06	1.369e-05	4.134e-06	1.26e-09	1.09e-09
Adam+L-BFGS-B	4.556e-07	1.247e-06	3.894e-07	2.33e-10	2.39e-10

of tanh produced the best approximation with the lowest possible margin of error, while applying the ReLu activation function yields the worst result compared to the others.

Discussion on the selection of neural network design and hidden layer Choosing the appropriate neural network architecture is one of the parameters that affect the training success of PINNs [70]. Here, we use seven different neural network layouts during the model construction process to examine the impact of neural network size on the performance of our models. The same five hidden

layers were utilized in every training. We assumed that the neural network structure is composed of one output layer, which represents the prediction result, and one input layer, which has two nodes. As can be observed from Table 3 and Fig. 5a, in terms of L_2 , L_∞ , RMSE, test loss, and train loss the neural network architecture $2 + 100(5) + 1$ gives smaller error values. The neural network structures with 7, 10, and 25 nodes produce high errors compared to the others. The neural networks with nodes 50, 80, and 120 have a uniform prediction that is almost near to the one with 100 node results. Additionally, using the neural network architecture having 100 nodes, the comparison between several hidden layers was performed as reported in Fig. 5b and Table 4. The outcomes show that the optimal prediction was obtained when we used 5 hidden layers.

Discussion on the selection of learning rate The choice of an adequate learning rate is essential to the effective use of PINNs. The learning rate defines the step size throughout the optimization process, and the choice can have a substantial impact on the model's convergence and performance [71, 72]. The setting of the learning rate for

Table 2 PINNs approximation errors and losses for different activation functions based on Adam+LBFGS-B optimization

Activation function	L_2	L_∞	RMSE	Train loss	Test loss
tanh	4.556e-07	1.247e-06	3.894e-07	2.33e-10	2.39e-10
Sigmoid	2.431e-05	2.110e-05	2.942e-06	3.57e-09	3.62e-09
ReLu	5.544e-05	4.330e-04	5.301e-05	8.93e-08	8.97e-08
Sin	1.534e-05	1.021e-04	1.431e-06	1.09e-08	1.29e-08
Swish	2.401e-05	6.751e-05	2.435e-06	1.93e-08	2.01e-08
eLu	5.520e-05	4.263e-04	5.290e-05	7.320e-08	7.98e-08

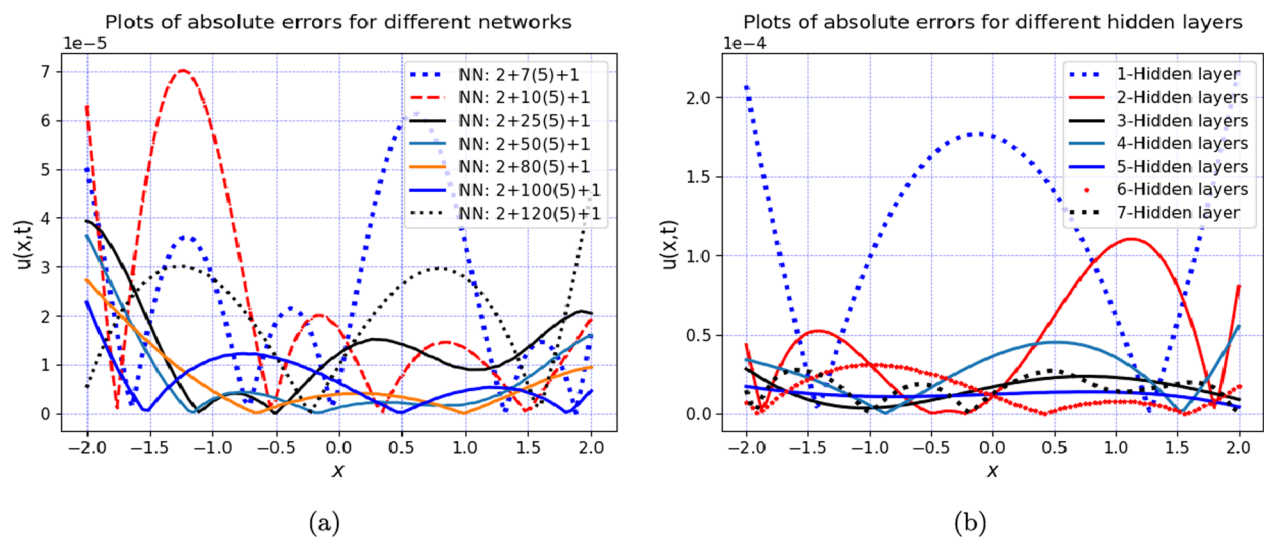


Fig. 5 The line plots of PINNs absolute error for different **a** neural network structures at $t = 0.5$ and **b** hidden layers at $t = 3.5$

Table 3 PINNs approximation errors along with train and test losses for various neural network structure nodes

Neural networks structure	L_2	L_∞	RMSE	Train loss	Test loss
2+7(5)+1	3.981e-05	9.198e-05	3.617e-05	1.24e-08	5.01e-08
2+10(5)+1	3.805e-05	1.109e-04	3.601e-05	1.09e-08	8.79e-09
2+25(5)+1	1.976e-05	5.941e-05	1.853e-06	3.42e-09	2.86e-09
2+50(5)+1	2.562e-05	6.258e-05	2.488e-05	7.38e-10	7.45e-10
2+80(5)+1	2.630e-05	6.025e-05	2.584e-05	3.72e-10	3.66e-10
2+100(5)+1	1.840e-05	5.219e-06	5.219e-06	2.39e-10	2.33e-10
2+120(5)+1	3.461e-05	8.183e-05	3.246e-05	5.27e-09	5.97e-09

Table 4 PINNs approximation errors along with train and test losses for different hidden layers

Hidden layer	L_2	L_∞	RMSE	Train loss	Test loss
1	1.137e-04	2.661e-04	1.138e-04	1.18e-07	1.49e-07
2	8.028e-05	2.110e-04	7.01e-05	6.09e-08	5.87e-08
3	1.218e-05	3.527e-05	1.269e-05	9.53e-10	1.13e-09
4	3.382e-05	8.794e-05	3.382e-05	1.00e-09	1.04e-09
5	7.499e-06	7.233e-06	1.720e-05	8.27e-10	8.58e-10
6	2.116e-05	5.185e-05	1.919e-05	2.58e-09	2.74e-09
7	4.596e-05	1.262e-04	4.409e-05	1.26e-09	1.08e-09

PINNs is a delicate balance that must take into account the interaction of physical limits and data-driven learning. Neural networks trained with physical restrictions may require different learning rates than typical machine learning models, as added physical knowledge might

Table 5 PINNs approximation errors along with train and test losses for different learning rates

Activation functions	L_2	L_∞	RMSE	Train loss	Test loss
1e-01	1.637e-02	7.103e-02	1.577e-02	2.85e-09	8.82e-05
1e-02	2.58e-02	2.867e-02	2.867e-03	1.76e-06	1.66e-06
1e-03	8.028e-05	2.110e-04	7.01e-05	2.33e-10	2.39e-10
2e-03	1.463e-04	1.753e-04	1.435e-04	2.50e-09	2.76e-09
3e-03	3.023e-03	3.050e-03	1.371e-04	2.91e-09	2.94e-09
4e-03	5.542e-03	5.562e-03	5.541e-03	1.04e-07	1.27e-07
5e-03	8.303e-03	8.316e-03	8.315e-04	3.45e-08	2.73e-08
6e-03	3.32e-03	3.41e-03	3.067e-03	2.97e-09	2.98e-09
7e-03	3.921e-03	3.940e-03	3.920e-03	7.53e-08	7.41e-08
8e-03	1.992e-02	1.994e-02	1.992e-03	2.92e-09	2.92e-09
9e-03	2.223e-02	2.231e-02	2.222e-02	8.17e-07	1.23e-06
1e-04	6.158e-04	1.414e-03	6.115e-04	1.04e-07	1.27e-07
1e-05	5.687e-04	1.849e-03	5.752e-04	2.95e-07	3.62e-07
1e-06	5.270e-04	1.734e-03	5.249e-04	3.80e-07	4.78e-07
1e-07	2.778e-01	3.0192e-01	2.778e-02	2.95e-07	3.62e-07

complicate the optimization landscape. For this work, we have performed various trainings for different learning rates, and the results are recorded in Table 5. The results reveal that for the learning rate $1e - 07$, the worst prediction was obtained. The best prediction was found for the learning rate $1e - 03$.

Discussion on the selection of training data set PINNs rely primarily on physics-based loss, yet the distribution of training points still influences their capacity to generalize across the domain. Underfitting occurs when there are too few training points, and the PINNs fail to adequately

reflect the underlying physical activity. The model may struggle to meet the governing equations (for example, PDEs) or boundary/initial conditions. Furthermore, when employing fewer training points, the model optimizer may have difficulties finding a solution that fulfills the physics. This means that convergence may become unstable, particularly in locations with sparse sampling. Too many training points can result in decreased accuracy, which goes back because PINNs are essentially

Table 6 PINNs approximation errors along with train and test losses for different training points

Training sample	L_2	L_∞	RMSE	Train loss	Test loss
100	8.455e-05	1.788e-04	8.104e-05	2.30e-08	2.60e-08
200	3.828e-05	7.238e-05	2.823e-05	6.73e-09	8.65e-09
300	1.169e-05	3.098e-05	1.092e-05	2.17e-09	2.50e-09
400	1.183e-05	2.837e-05	1.129e-05	1.74e-09	1.92e-09
500	6.215e-05	8.342e-05	6.472e-06	9.58e-09	1.01e-08
600	1.294e-05	3.697e-05	1.153e-05	1.50e-09	1.49e-09
700	3.745e-05	6.052e-05	3.746e-06	8.76e-09	9.69e-09
800	2.418e-05	5.974e-05	2.411e-06	1.85e-09	1.81e-09
900	2.405e-05	5.712e-05	2.405e-06	2.87e-09	3.14e-09
1000	2.124e-05	4.905e-05	1.852e-05	6.57e-09	6.38e-09
1100	7.426e-05	8.561e-05	7.426e-06	9.24e-10	9.00e-10
1200	4.253e-05	9.261e-05	4.049e-05	3.11e-09	2.78e-09
1300	3.224e-05	4.919e-05	3.226e-06	8.35e-09	8.38e-09
1400	1.520e-05	3.334e-05	1.520e-06	2.04e-09	1.93e-09
1500	5.734e-05	1.622e-04	7.655e-06	2.19e-09	2.14e-09
1600	1.332e-05	2.000e-05	1.332e-06	3.92e-09	4.22e-09
1700	4.861e-06	1.469e-05	4.861e-07	5.22e-09	5.27e-09
1800	9.205e-06	7.795e-06	1.097e-06	5.86e-09	6.01e-09
1900	6.040e-06	2.390e-05	2.7848e-06	1.09e-09	1.07e-09
2000	6.114e-07	1.526e-06	5.976e-07	4.12e-10	4.17e-10

constrained by physics rather than data. If unnecessary training points are supplied, the computational overhead rises without yielding considerable progress. As a result, selecting a suitable amount of data points requires careful attention. Sufficient training points increase accuracy by sampling the solution space more thoroughly and capturing complex solution properties such as sharp gradients or discontinuities. Also improves the PINN’s capacity to generalize across domains. Table 6 shows that employing more training points improves accuracy and convergence. In Table 6, we undertake several experiments and obtain good approximations from data points ranging from 100 to 2000. Below 100 training points and over 2000 points, the model produces erroneous predictions, which we excluded from the reports. However, between 100 and 2000 samples used for training, the accuracy of the results improves as the number of training samples increases, with 2000 training samples yielding the ideal solution.

Remark 1 The optimal hyperparameters acquired from varied training were used for each example’s solution graphing and associated simulation analysis.

The solution plotting to Example 4.1

The solution plots of the PINNs result with the precise answer for the problem (32) is provided in Figs. 6, 7, 8. The precise solutions in Figs. 6b and 7b and the predicted solution in Figs. 6a and 7a are correspondingly almost identical. In Fig. 8a the plots of PINN’s estimation and the exact solutions overlap. There is a small error that is much closer to zero between the two solutions, as shown by the absolute error 3D Fig. 6c and 2D Fig. 7c, which further confirms the great performance of the offered method. Additionally, the absolute errors for various

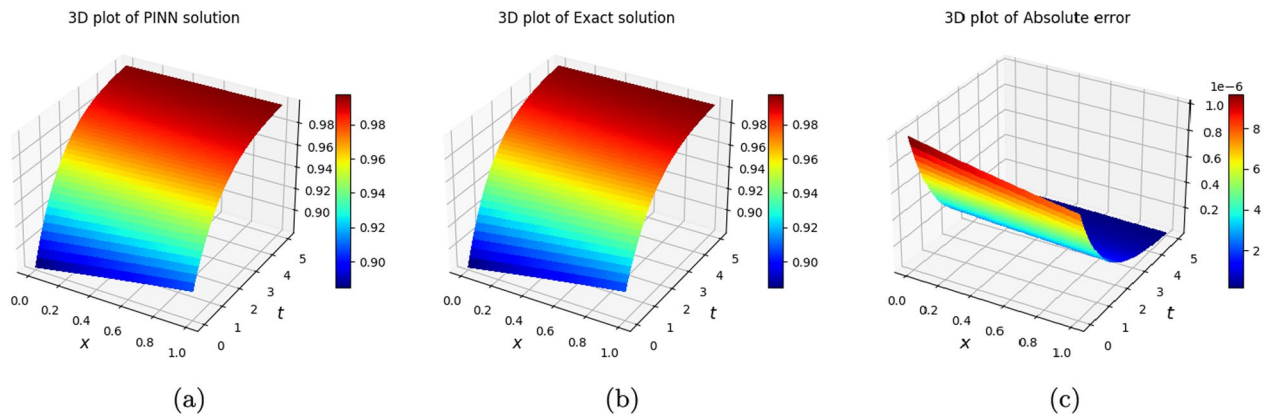


Fig. 6 3D visualizations of the **a** PINNs outcome, **b** exact answer, and **c** corresponding absolute error to Dirichlet BCs problem (32)

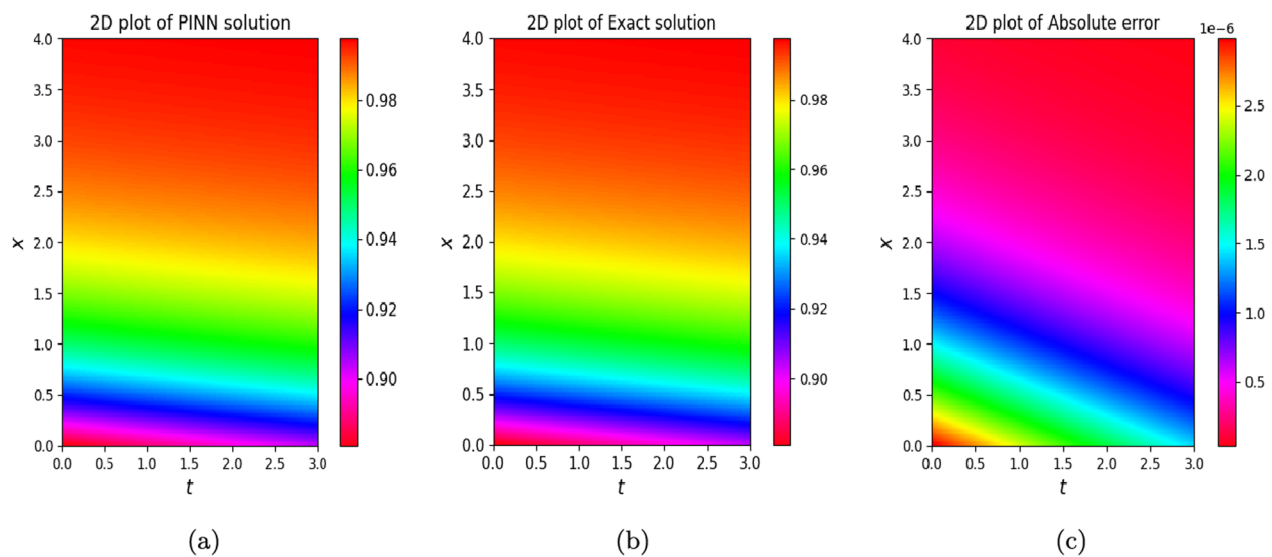


Fig. 7 2D plots of the Dirichlet BCs problem (32) with the **a** PINNs outcome, **b** True solution, and **c** corresponding absolute error

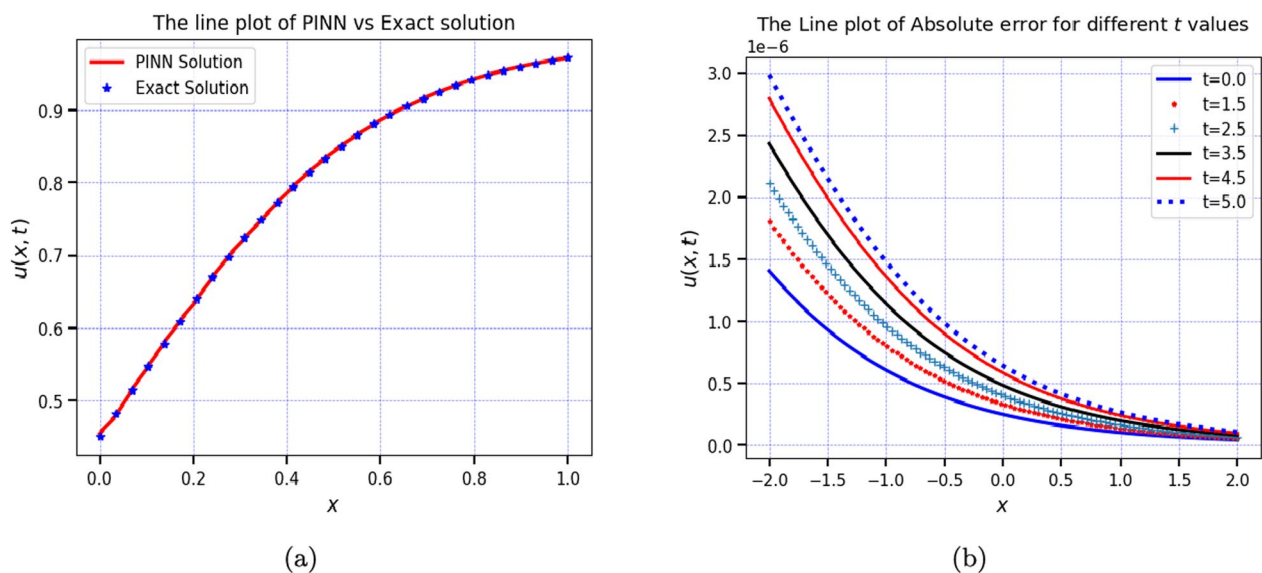


Fig. 8 Line plots showing **a** the difference between the actual solution and PINNs prediction at $t = 3.5$ and **b** the associated absolute error for different t values to problem (32)

time variables t were presented in Table 7 and line plots in Fig. 8b. The results indicate that the proposed model provides a superior answer at $t = 0$ and that the accuracy of the result decreases as t gets larger.

Statistical error analysis to Example 4.1

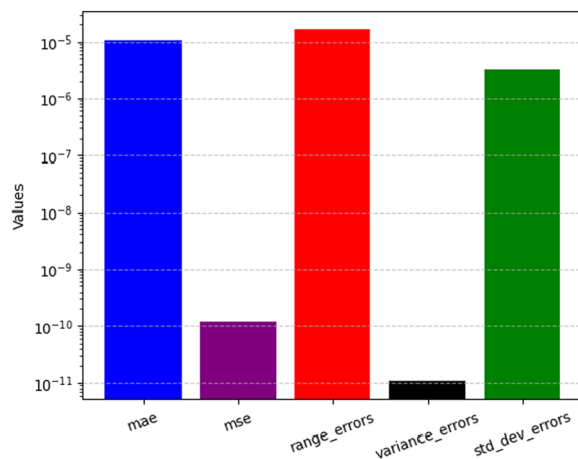
In Table 7 and Fig. 7b based on the well-known error metrics such as L_2 , L_∞ and RMSE, the accuracy of the proposed deep learning approach was examined. To further explore the effectiveness and generalization

Table 7 The comparison of PINNs results with the exact solution for different values of time variable t to Example 4.1

t	L_2	L_∞	RMSE
0.0	6.114×10^{-7}	1.526×10^{-6}	5.976×10^{-7}
1.5	7.334×10^{-7}	1.800×10^{-6}	7.134×10^{-7}
2.5	8.764×10^{-7}	2.107×10^{-6}	8.476×10^{-7}
3.5	1.033×10^{-6}	2.429×10^{-6}	9.931×10^{-7}
4.5	1.223×10^{-6}	2.793×10^{-6}	1.165×10^{-6}
5.0	1.328×10^{-6}	2.982×10^{-6}	1.259×10^{-6}

Table 8 Statistical error analysis of PINNs results for different values of time variable t to Example 4.1

t	MAE	MSE	Range	Variance	Standard deviation
0.0	1.010e-05	1.207e-10	1.610e-05	4.870e-12	3.803e-06
0.1	1.128e-05	1.312e-10	1.875e-05	1.822e-11	4.269e-06
0.2	1.128e-05	1.386e-10	1.830e-05	1.965e-11	4.269e-06
0.3	1.184e-05	1.464e-10	1.896e-05	2.091e-11	4.433e-06
0.4	1.266e-05	1.617e-10	2.049e-05	2.257e-11	4.582e-06
0.5	1.341e-05	1.933e-10	2.256e-05	2.408e-11	4.823e-06
0.6	1.429e-05	2.029e-10	2.411e-05	2.575e-11	5.075e-06
0.7	1.497e-05	2.179e-10	2.642e-05	2.724e-11	5.309e-06
0.8	1.581e-05	2.339e-10	2.851e-05	2.883e-11	5.617e-06
0.9	1.581e-05	2.648e-10	2.942e-05	3.079e-11	5.689e-06
1.0	1.736e-05	2.716e-10	3.005e-05	3.237e-11	5.723e-06

**Fig. 9** The graphical representation of the statistical error analysis for PINNs approximation to Example 4.1

ability of PINNs in approximating the Dirichlet boundary value problem (32), we present different statistical performance measures, including MAE, MSE, range error, variance error, and standard deviation error. The experiments based on those error metrics, are recorder in Table 8 and shown graphically in Fig. 9. In Fig. 9 the black bar represents variance errors, which are the smallest (about 10^{-11}). It determines how effectively the predicted solution captures the variability of the exact solution. The variations in errors are almost less than or equal to 10^{-11} that, which shows the predicted solutions are stable and correct. Our suggested approach is highly dependable and efficient, as seen by the extremely minimal mistakes we obtained across all statistical error indicators.

Example 4.2 Nonlinear telegraph equation with periodic BCs

In the second test, we considered the following nonlinear telegraph equation:

$$\frac{\partial^2 u}{\partial t^2} + 4 \frac{\partial u}{\partial t} + u^3 = \frac{\partial^2 u}{\partial x^2} - 2e^{-t} \sin(x) + e^{-3t} \sin^3(x), \quad (35)$$

$$0 \leq x \leq 2\pi, 0 \leq t \leq 0.5,$$

with the initial conditions

$$u(x, 0) = \sin(x), 0 \leq x \leq 2\pi, \quad (36a)$$

$$u_t(x, 0) = -\sin(x), 0 \leq x \leq 2\pi, \quad (36b)$$

and boundary conditions

$$u(0, t) = 0, 0 \leq t \leq 0.5, \quad (37a)$$

$$u(2\pi, t) = 0, 0 \leq t \leq 0.5, \quad (37b)$$

The analytical solution to BVP (35)–(37) is given by $u(x, t) = e^{-t} \sin(x)$ [73]. The recommended neural-network design is trained using the suggested approach to minimize the loss function (28) using the procedures described in Section 3 to resolve the problem 4.2.

The solution plotting to Example 4.2

The PINNs result to problem (35) along with the exact answer is shown in Figs. 10, 11, 12 of 3D, 2D, and line plots. As we can see from the figures, there is little difference between the two outcomes, which reveals that they are adequately connected. The absolute error between the exact answer and the proposed method presented in 3D Fig. 10c and 2D Fig. 10c shows the suggested technique gives good prediction with an error much closer to zero, which can be also validated by the overlap line plots of the exact and predicted solution presented in Fig. 12a. The absolute error between the two solutions is also examined in Table 9 and Fig. 12b for time variable $t = 0.0, 1.0, 2.0, 3.5, 4.5, \& 6.0$. The results show that the error between the two solutions is almost zero. The maximum error is in the order of $10e-5$ which occurs at $t = 0.5$ near the final time. For the time variable $t = 0$ and near 0 we get the minimum absolute error. For the space variable x we obtain the maximum error at the boundaries $x = 0$ and $x = 2\pi$ and almost the same result between 0 and 2π .

Statistical error analysis to Example 4.2

To verify the effectiveness of PINNs in approximating the periodic boundary value problem (35), we provide statistical performance metrics like MAE, MSE, range error, variance error, and standard deviation, similar to those used in

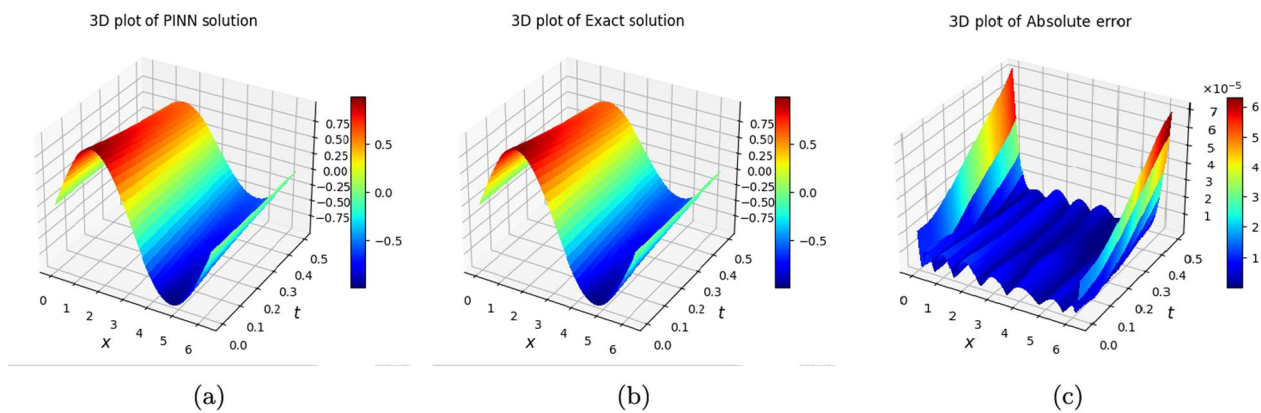


Fig. 10 3D graphs of the **a** predicted solution via PINNs, **b** actual answer, and **c** corresponding absolute error to the periodic BCs case of problem (35)

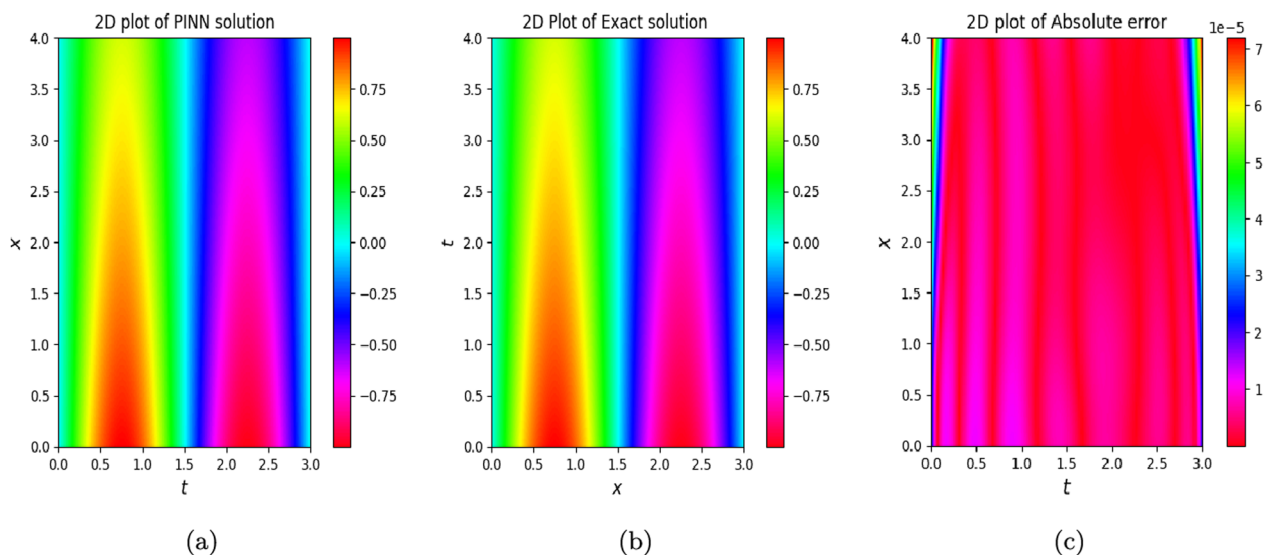


Fig. 11 2D plots of the periodic BCs problem (35) with the **a** PINNs outcome, **b** True solution, and **c** corresponding absolute error

Example 4.1. As in Table 10 and Fig. 13, the typical magnitude of prediction errors indicated by the MAE maximum value is approximately 10^{-5} . This implies that the absolute prediction errors of the model are generally quite low. The average of the squared discrepancies between expected and actual values is measured by the MSE. Its much smaller number (around 10^{-10}) suggests that no notable outliers are causing greater squared errors and that the majority of errors are modest. Similarly, the indicated technique ability and trustworthiness are revealed by the remaining metrics shown in Table 10 and Fig. 13.

Example 4.3 Nonlinear telegraph equation with Neumann BCs

In the third test, we examined the nonlinear space-time telegraph equation in the manner described below:

$$\frac{\partial^2 u}{\partial t^2} + 2 \frac{\partial u}{\partial t} + u^2 = \frac{\partial^2 u}{\partial x^2} - 2e^{-t} \cosh(x) + e^{-2t} \cosh^2(x), \quad (38)$$

$$-2 \leq x \leq 2, 0 \leq t \leq 0.5,$$

with the initial conditions

$$u(x, 0) = \cosh(x), \quad -2 \leq x \leq 2, \quad (39a)$$

$$u_t(x, 0) = -\cosh(x), \quad -2 \leq x \leq 2, \quad (39b)$$

and Neumann boundary conditions

$$u_x(-2, t) = -e^{-t} \sinh(2), \quad 0 \leq t \leq 0.5, \quad (40a)$$

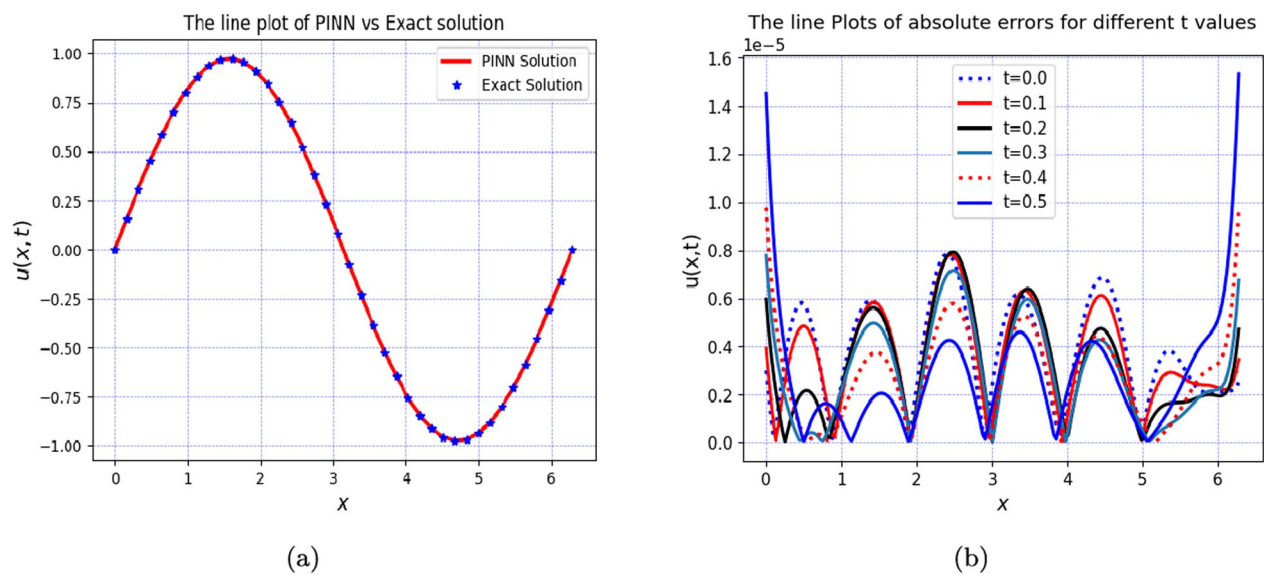


Fig. 12 The line graphs of the problem (35) show **a** the PINNs-approximation vs exact answer at $t = 4.0$ and **b** the associated absolute error for various t values

Table 9 The comparison of PINNs results with the exact solution for different values of time variable t to Example 4.2

t	L_2	L_∞	RMSE
0.0	6.051×10^{-6}	7.827×10^{-6}	4.268×10^{-6}
0.1	6.250×10^{-6}	7.816×10^{-6}	4.088×10^{-6}
0.2	6.248×10^{-6}	7.921×10^{-6}	3.790×10^{-6}
0.3	6.327×10^{-6}	7.775×10^{-6}	3.559×10^{-6}
0.4	6.497×10^{-6}	9.776×10^{-6}	3.390×10^{-6}
0.5	7.9402×10^{-6}	1.533×10^{-5}	3.746×10^{-6}

Table 10 Statistical error analysis of PINNs results for different values of time variable t to Example 4.2

t	MAE	MSE	Range	Variance	Standard deviation
0.0	5.291e-06	9.161e-11	1.467e-05	3.905e-12	2.588e-06
0.1	5.571e-06	9.268e-11	2.114e-05	4.231e-12	2.620e-06
0.2	5.807e-06	9.439e-11	2.127e-05	4.547e-12	2.613e-06
0.3	5.968e-06	9.541e-11	2.130e-05	5.101e-12	2.674e-06
0.4	6.132e-06	9.418e-11	2.150e-05	5.145e-12	2.715e-06
0.5	6.978e-06	9.588e-11	2.853e-05	6.015e-12	2.757e-06
0.6	7.430e-06	9.641e-11	2.867e-05	7.823e-12	2.782e-06
0.7	7.907e-06	9.525e-11	3.045e-05	7.932e-12	2.858e-06
0.8	8.231e-06	9.669e-11	3.116e-05	8.807e-12	2.887e-06
0.9	9.305e-06	9.670e-11	3.540e-05	8.964e-12	3.341e-06
1.0	9.796e-06	9.688e-11	3.617e-05	9.731e-12	3.518e-06

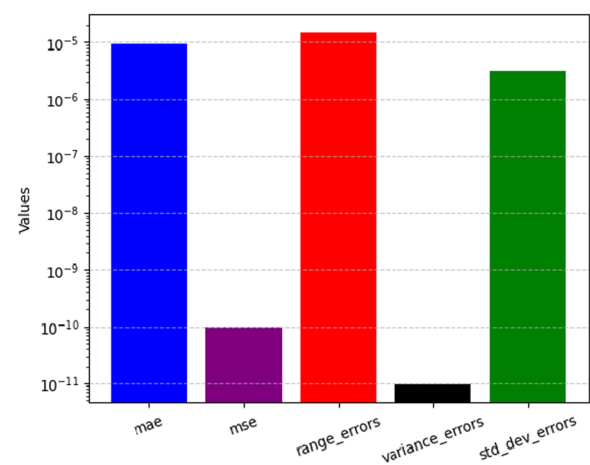


Fig. 13 The graphical representation of the statistical error analysis for PINNs approximation to Example 4.2

$$u_x(2, t) = e^{-t} \sinh(2), \quad 0 \leq t \leq 0.5. \quad (40b)$$

The exact solution of BVP (38) is given by $u(x, t) = e^{-t} \cosh(x)$ [74]. To solve the problem (38) of Example 4.3, the proposed neural network design is trained using the suggested method to minimize the loss function (24) following the steps outlined in Section 3.

The solution plotting to Example 4.3

The 3D Fig. 14a of PINNs and the 3D Fig. 14b of the precise solution to Equation (38) demonstrate their near equivalency. Likewise, the two solutions' 2D

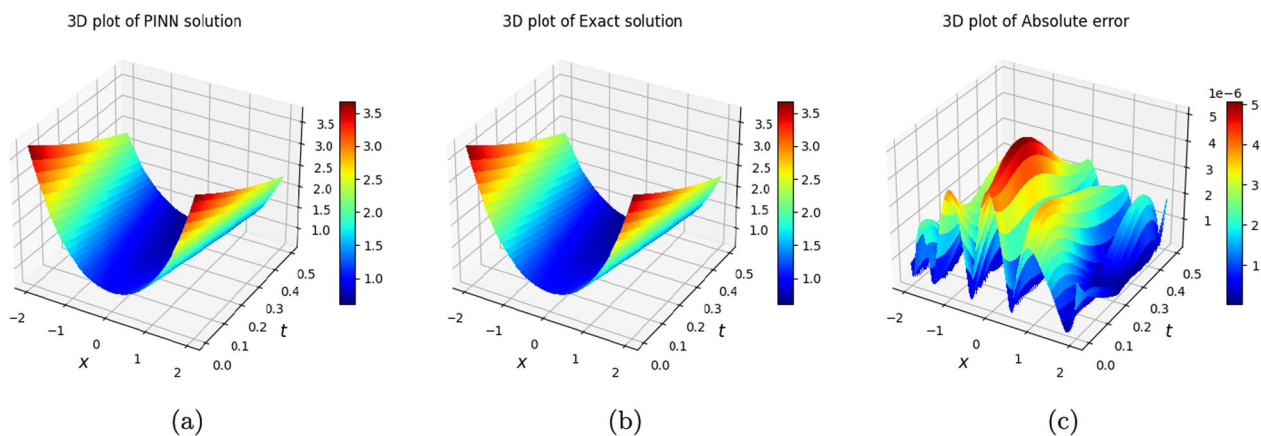


Fig. 14 3D visualizations of the **a** outcome gained via PINNs, **b** exact answer, and **c** corresponding absolute error to Neumann BCs problem (38)

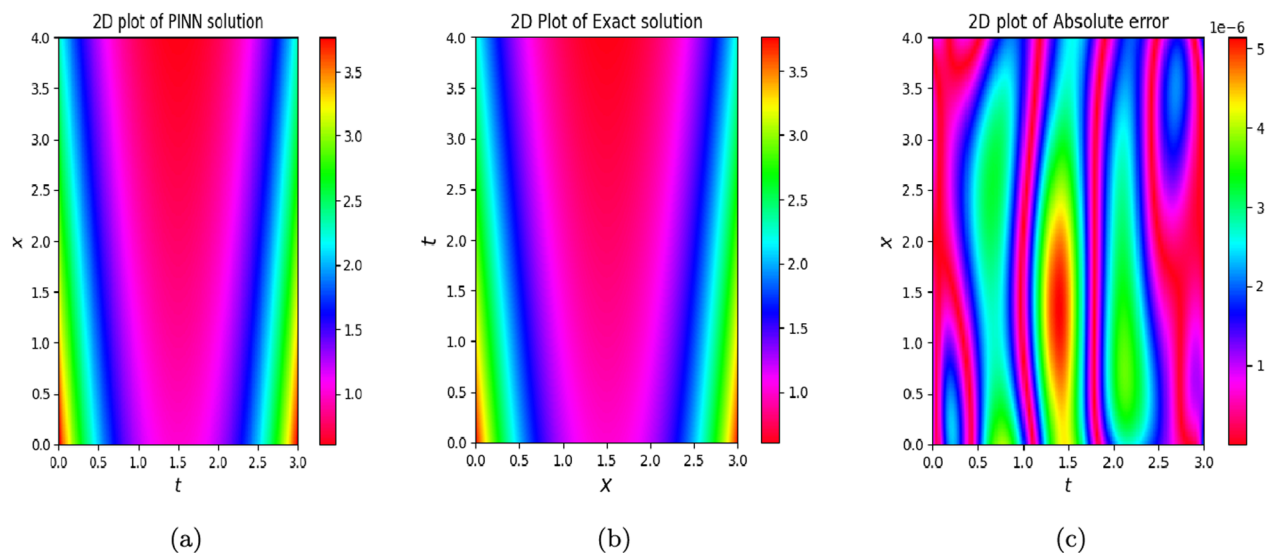


Fig. 15 2D sketch of the Neumann BCs problem (38) with the **a** PINNs outcome, **b** True solution, and **c** corresponding absolute error

Figs. 15a and 15b are identical. Therefore, the suggested method provides a good solution that matches the precise response. Additionally, the line diagrams of the two solutions coincide in Fig. 16a, indicating that they are potentially closely related. The proposed method is highly efficient and powerful in approximating the given non-linear problem, as demonstrated by the comparisons between its solution and the exact solution regarding absolute errors in 2D Fig. 15c and 3D Fig. 14c. The absolute error between the solutions is in the order of $10e - 6$. Fig. 16b and Table 11 compare the exact solution and the PINNs predicted for various time variables. It demonstrates that the best prediction is made at $t = 0.0$. Additionally, for $t = 0.1$ and $t = 0.7$, we achieve a decent

solution in comparison to the other times. Furthermore, Fig. 14c and Fig. 16b show that, for the space variable x , the two boundaries $x = -2.0$ and $x = 2.0$ yield the best prediction, with the largest absolute error near the origin at $x = 0$.

Statistical error analysis to Example 4.3

We offer some statistical performance metrics like those employed in Examples 4.1 and 4.2 in order to confirm the efficacy of PINNs in approximating the Neumann boundary value issue (38). As we can observe from Table 12 and Fig. 17, compared to the Dirichlet and periodic boundary conditions problems presented in Examples 4.1 and 4.2, PINN is more efficient for Neumann boundary

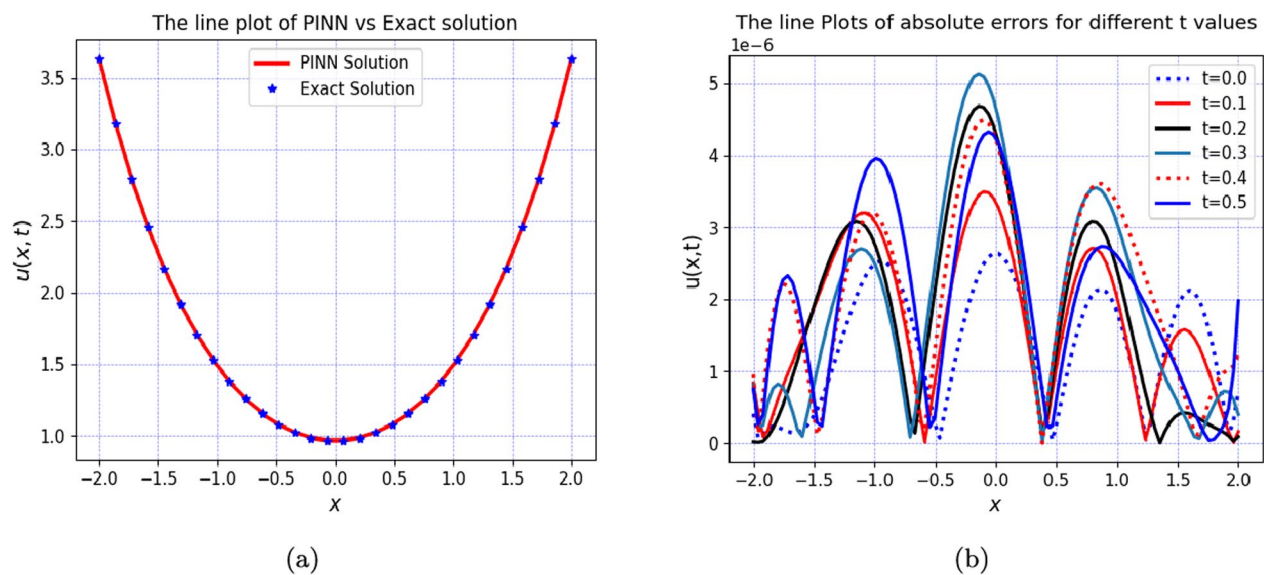


Fig. 16 The line graphs of the problem (38) show **a** the PINNs-approximation vs exact answer at $t = 2.0$ and **b** the associated absolute error for various t values

Table 11 The comparison of PINNs results with the exact solution for different values of time variable t to Example 4.3

t	L_2	L_∞	RMSE
0.0	2.380×10^{-7}	9.994×10^{-7}	7.124×10^{-7}
0.1	2.585×10^{-7}	1.974×10^{-6}	7.735×10^{-7}
0.2	1.881×10^{-6}	3.188×10^{-6}	2.691×10^{-6}
0.3	3.251×10^{-6}	3.867×10^{-6}	2.779×10^{-6}
0.4	1.046×10^{-6}	2.501×10^{-6}	1.496×10^{-6}
0.5	4.556×10^{-7}	1.247×10^{-6}	3.894×10^{-7}

Table 12 Statistical error analysis of PINNs results for different values of time variable t to Example 4.3

t	MAE	MSE	Range	Variance	Standard deviation
0.0	9.689e-07	9.205e-12	1.105e-06	2.106e-14	7.122e-07
0.1	8.643e-07	9.213e-12	1.119e-06	4.995e-13	7.122e-07
0.2	8.120e-07	9.34e-12	1.137e-06	5.072e-13	7.133e-07
0.3	9.689e-07	9.261e-12	1.408e-06	5.350e-13	7.067e-07
0.4	1.073e-06	9.309e-12	1.747e-06	5.848e-13	7.404e-07
0.5	1.483e-06	9.674e-12	1.788e-06	6.200e-13	7.723e-07
0.6	2.024e-06	9.870e-12	1.949e-06	6.964e-13	9.039e-07
0.7	3.105e-06	9.887e-12	2.002e-06	8.171e-13	9.658e-07
0.8	3.271e-06	9.991e-12	2.158e-06	9.329e-13	9.207e-07
0.9	3.490e-06	9.982e-12	2.593e-06	9.017e-13	9.496e-07
1.0	4.106e-06	9.997e-12	3.360e-06	9.774e-13	9.886e-07

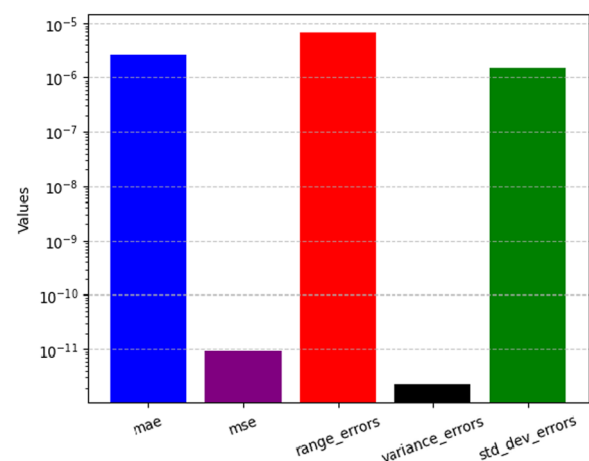


Fig. 17 The graphical representation of the statistical error analysis for PINNs approximation to Example 4.3

conditions in Example 4.3. Our research revealed that the suggested approach yields good outcomes across all error indicators. Thus, it can be concluded that PINNs are a machine-learning technique that is effective, dependable, accurate, and easily adjustable and may be used to address a variety of real-world issues.

Conclusions and prospects

In order to solve a hyperbolic non-linear telegraph equation, we have introduced an artificial intelligence framework-based algorithm in this paper termed PINNs. We gave PINNs a multi-objective cost function that includes

the initial condition, boundary constraints, and controlling PDE residual over randomly allocated collocation points across the problem-defined area in order to solve the suggested scenario effectively. To show how effectively the proposed model worked, we examined the proposed problem concerning boundary conditions of Dirichlet, periodic, and Neumann as benchmark instances. Using the Python software, we executed several trials and manipulated tables and graphs to mimic the outcomes. The trial conducted on the selection of the best optimization technique demonstrates that the L-BFGS-B algorithm outperforms the Adam, SGD, and RMSprop algorithm strategies. However, merging Adam and L-BFGS-B yields the most effective results. The optimal activation function used with the proposed model is also identified by analyzing six different activation functions. According to what was discovered, the *tanh* function with neural network architecture having 5 hidden layers and 100 nodes per hidden layer gives the most accurate results, while the poorest result is achieved via the ReLU activation function. A 0.001 learning rate is found to be appropriate for the suggested PINNs model based on experimental training on the choice of learning rate. Regarding the experiments on the number of collocation points or training samples, the findings suggested that for data sets near sufficient, as the number of training points increases, the model accuracy also increases. However, taking into account too few or too many collocation sites might result in an unstable solution, overfitting, or increased computing complexity. The error analysis between the exact and PINNs estimation is examined in terms of relative error metrics such as L_∞ , L_2 , and RMSE and recorded by utilizing graphs and tables. We have also performed various extensive statistical analyses based on different errors like range, variance, and standard deviation. With the variation being very near to zero, the findings demonstrate that the suggested approach can correctly determine the appropriate approximation to the nonlinear space-time hyperbolic telegraph equation considering various boundary conditions. of the data necessary for this research report.

Author contributions

ATD formulated the problem, designed, and drafted the manuscript. ASB carried out scheme development, numerical experimentation and produced tables and figures. All authors read and approved the final manuscript.

Funding

There is no any fund support in this research work.

Data availability

Not applicable.

Declarations

Ethics approval and consent to participate

Not applicable.

Consent for publication

Not applicable.

Competing interests

The authors declare that they have no competing interests.

Received: 17 November 2024 Accepted: 7 February 2025

Published online: 19 February 2025

References

1. Scott AC. Physical applications of nonlinear theory. *The Nonlinear Universe: Chaos, Emergence, Life*. 2007;p. 101–179.
2. Butt ZI, Ahmad I, Shoaib M, Ilyas H, Raja MAZ. Electro-magnetohydrodynamic impact on Darcy-Forchheimer viscous fluid flow over a stretchable surface: integrated intelligent neuro-evolutionary computing approach. *Int Commun Heat Mass Transf*. 2022;137: 106262.
3. Konane D, Ouedraogo WYSB, Guingane TT, Zongo A, Koalaga Z, Zougmore F. An exact solution of telegraph equations for voltage monitoring of electrical transmission line. *Energy and Power Eng*. 2022;14(11):669–79.
4. Abrori M, Sugiyanto S, Sari HMS. Double Laplace transform method for solving telegraph equation. *JP J Heat Mass Transf*. 2019;17(1):265–275.
5. Hussien S, Uddin M, Karim MR. An efficient computational technique for the analysis of telegraph equation. *J Eng Adv*. 2022;3(3):104–11.
6. Dubey S, Chakraverty S, Kundu M. Approximate solutions of space and time fractional telegraph equations using Taylor series expansion method. *J Comput Anal Appl*. 2023;31(1):48.
7. Raslan K, K Ali K, Al-Jeaid HK, Shaalan M. Bi-finite difference method to solve second-order nonlinear hyperbolic telegraph equation in two dimensions. *Math Probl Eng*. 2022;2022(1):1782229.
8. Heydari M, Razzaghi M, Karami S. Orthonormal Chelyshkov polynomials for multi-term time fractional two-dimensional telegraph type equations. *Results Phys*. 2023;55: 107161.
9. Alaroud M, Alomari AK, Tahat N, Al-Omari S, Ishak A. A novel solution approach for time-fractional hyperbolic telegraph differential equation with caputo time differentiation. *Mathematics*. 2023;11(9):2181.
10. Deresse AT. Analytical solutions to two-dimensional nonlinear telegraph equations using the conformable triple laplace transform iterative method. *Adv Math Phys*. 2022;2022(1):4552179.
11. Zeng J, Idrees A, Abdo MS. A new strategy for the approximate solution of hyperbolic telegraph equations in nonlinear vibration system. *J Funct Spaces*. 2022;2022(1):8304107.
12. Sahraee Z, Arabameri M. A semi-discretization method based on finite difference and differential transform methods to solve the time-fractional telegraph equation. *Symmetry*. 2023;15(9):1759.
13. Sabdin ARF, Hussin CHC, Ekal GB, Mandangan A, Sulaiman J. Approximate analytical solution for time-fractional nonlinear telegraph equations with source term. *J Adv Res Appl Sci Eng Technol*. 2023;31(1):132–43.
14. Mamadu E, Ojarikre H, Njoseh I. An error analysis of implicit finite difference method with Mamadu-Njoseh basis functions for time fractional telegraph equation. *Asian Res J Math*. 2023;19(7):20–30.
15. Korzyuk V, Rudzko J. Classical solution of the second mixed problem for the telegraph equation with a nonlinear potential. *Differ Equ*. 2023;59(9):1216–34.
16. Karakostas X, Caviedes-Nozal D, Richard A, Fernandez-Grande E. Room impulse response reconstruction with physics-informed deep learning. *J Acoust Soc Am*. 2024;155(2):1048–59.
17. Saviolo A, Li G, Loianno G. Physics-inspired temporal learning of quadrotor dynamics for accurate model predictive trajectory tracking. *IEEE Robot Autom Lett*. 2022;7(4):10256–63.
18. Chen Z, Lai SK, Yang Z. AT-PINN: advanced time-marching physics-informed neural network for structural vibration analysis. *Thin-Walled Struct*. 2024;196: 111423.

19. Yadav N, Yadav A, Kumar M, et al. An introduction to neural network methods for differential equations. vol. 1. Springer; 2015.
20. Livingstone DJ. Artificial neural networks: methods and applications. vol. 458. Springer; 2008.
21. Chakraborty S, Mall S. Artificial neural networks for engineers and scientists: solving ordinary differential equations. CRC Press; 2017.
22. Lagaris IE, Likas A, Fotiadis DI. Artificial neural networks for solving ordinary and partial differential equations. *IEEE Trans Neural Netw*. 1998;9(5):987–1000.
23. Khan NA, Hussain S, Spratford W, Goecke R, Kotecha K, Jamwal PK. Deep learning-driven analysis of a six-bar mechanism for personalized gait rehabilitation. *J Comput Inf Sci Eng*. 2024;25: 011001.
24. Khan NA, Laouini G, Alshammari FS, Khalid M, Aamir N. Supervised machine learning for jamming transition in traffic flow with fluctuations in acceleration and braking. *Comput Electr Eng*. 2023;109: 108740.
25. Khan NA, Sulaiman M, Lu B. Predictive insights into nonlinear nanofluid flow in rotating systems: a machine learning approach. *Eng Comput*. 2024;1–18. <https://doi.org/10.1007/s00366-024-01993-1>.
26. Butt ZI, Ahmad I, Shoaib M. Design of inverse multiquadric radial basis neural networks for the dynamical analysis of wire coating problem with Oldroyd 8-constant fluid. *AIP Adv*. 2022;12(10):105306. <https://doi.org/10.1063/5.0101601>.
27. Butt ZI, Ahmad I, Ilyas H, Shoaib M, Raja MAZ. Design of inverse multiquadric radial basis neural networks for the dynamical analysis of MHD cassin nanofluid flow along a nonlinear stretchable porous surface with multiple slip conditions. *Int J Hydrogen Energy*. 2023;48(42):16100–31.
28. Butt ZI, Ahmad I, Shoaib M, Ilyas H, Raja MAZ. A novel design of inverse multiquadric radial basis neural networks to analyze MHD nanofluid boundary layer flow past a wedge embedded in a porous medium under the influence of radiation and viscous effects. *Int Commun Heat Mass Transf*. 2023;140: 106516.
29. Butt ZI, Ahmad I, Shoaib M, Ilyas H, Kiani AK, Raja MAZ. Neuro-evolution heuristics for Prandtl-Eyring nanofluid flow with homogenous/heterogeneous reaction across a linearly heated stretched sheet. *Waves in Random and Complex Media*. 2023;p. 1–47.
30. Tang K, Wan X, Yang C. DAS-PINNs: a deep adaptive sampling method for solving high-dimensional partial differential equations. *J Comput Phys*. 2023;476: 111868.
31. Hvatov A, Aminev D, Demyanchuk N. Easy to learn hard to master-how to solve an arbitrary equation with PINN. In: *NeurIPS 2023 AI for Science Workshop*; 2023. p. 1–15.
32. Raissi M, Perdikaris P, Karniadakis GE. Physics informed deep learning (part i): Data-driven solutions of nonlinear partial differential equations. *arXiv preprint arXiv:1711.10561*. 2017.
33. Raissi M, Perdikaris P, Karniadakis GE. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J Comput Phys*. 2019;378:686–707.
34. Blechschmidt J, Ernst OG. Three ways to solve partial differential equations with neural networks—a review. *GAMM-Mitteilungen*. 2021;44(2): e202100006.
35. Schafer V. Generalization of physics-informed neural networks for various boundary and initial conditions. *Technische Universität Kaiserslautern*; 2022.
36. Deresse AT, Dufera TT. A deep learning approach: physics-informed neural networks for solving the 2D nonlinear Sine-Gordon equation. *Results Appl Math*. 2025;25: 100532.
37. Shin Y, Darbon J, Karniadakis GE. On the convergence of physics-informed neural networks for linear second-order elliptic and parabolic type PDEs. *arXiv preprint arXiv:2004.01806*. 2020.
38. Ma S, Zhang J, Shi C, Di P, Robertson ID, Zhang ZQ. Physics-informed deep learning for muscle force prediction with unlabeled sEMG signals. *IEEE Trans Neural Syst Rehabil Eng*. 2024;32:1246–56.
39. Liu H, Zhang Y, Wang L. Pre-training physics-informed neural network with mixed sampling and its application in high-dimensional systems. *J Syst Sci Complex*. 2024;37(2):494–510.
40. Vu-Quoc L, Humer A. Partial-differential-algebraic equations of nonlinear dynamics by physics-informed neural-network:(I) operator splitting and framework assessment. *Int J Numer Methods Eng*. 2024;2024: e7586.
41. Hettige KH, Ji J, Xiang S, Long C, Cong G, Wang J. AirPhyNet: harnessing physics-guided neural networks for air quality prediction. *arXiv preprint arXiv:2402.03784*. 2024.
42. Difonzo FV, Lopez L, Pellegrino SF. Physics informed neural networks for an inverse problem in peridynamic models. *Eng Comput*. 2024;1–10. <https://doi.org/10.1007/s00366-024-01993-1>.
43. Ibrahim W, Bijiga LK. Neural network method for solving time-fractional telegraph equation. *Math Probl Eng*. 2021;2021(1):7167801.
44. Babaei M, Mohammadi KM, Hajimohammadi Z, Parand K. JDNN: Jacobi Deep Neural Network for Solving Telegraph Equation. *arXiv preprint arXiv:2212.12700*. 2022.
45. Deresse AT, Dufera TT. Exploring physics-informed neural networks for the nonlinear generalized sine-Gordon equation. *Appl Comput Intell Soft Comput*. 2024;2024(1):3328977.
46. Karniadakis GE, Kevrekidis IG, Lu L, Perdikaris P, Wang S, Yang L. Physics-informed machine learning. *Nat Rev Phys*. 2021;3(6):422–40.
47. Rao C, Sun H, Liu Y. Physics-informed deep learning for computational elastodynamics without labeled data. *J Eng Mech*. 2021;147(8):04021043.
48. García-Cervera CJ, Kessler M, Perigo F. Control of partial differential equations via physics-informed neural networks. *J Optim Theory Appl*. 2023;196(2):391–414.
49. Michoski C, Milosavljević M, Oliver T, Hatch DR. Solving differential equations using deep neural networks. *Neurocomputing*. 2020;399:193–212.
50. Calin O. Deep learning architectures. Cham: Springer; 2020.
51. García Cabello J. Mathematical neural networks. *Axioms*. 2022;11(2):80.
52. Dufera TT. Deep neural network for system of ordinary differential equations: vectorized algorithm and simulation. *Mach Learn Appl*. 2021;5: 100058.
53. Kollmannsberger S, D'Angella D, Jokeit M, Herrmann L, et al. Deep learning in computational mechanics. Cham: Springer; 2021.
54. Abbasi J, Andersen PØ. Physical activation functions (PAFs): an approach for more efficient induction of physics into physics-informed neural networks (PINNs). *Neurocomputing*. 2024;608: 128352.
55. Sodhi SS, Chandra P. Bi-modal derivative activation function for sigmoidal feedforward networks. *Neurocomputing*. 2014;143:182–96.
56. Jagtap AD, Kawaguchi K, Em Karniadakis G. Locally adaptive activation functions with slope recovery for deep and physics-informed neural networks. *Proc R Soc A*. 2020;476(2239):20200334.
57. Kamali K. Deep Learning (Part 1)-Feedforward neural networks (FNN). *Galaxy Training Network*. 2024.
58. Skansi S, Skansi S. Feedforward neural networks. *Introduction to Deep Learning: From Logical Calculus to Artificial Intelligence*; 2018. p. 79–105.
59. Eldan R, Shamir O. The power of depth for feedforward neural networks. In: *Conference on learning theory*. PMLR; 2016. p. 907–940.
60. Lee S, Lee BT, Ko SK. A robust Gated-PINN to resolve local minima issues in solving differential algebraic equations. *Results Eng*. 2024;21: 101931.
61. Dixon MF, Halperin I, Bilokon P. Machine learning in finance. vol. 1170. Springer; 2020.
62. Dixon MF, Halperin I, Bilokon P, Dixon MF, Halperin I, Bilokon P. Feedforward neural networks. *Machine learning in finance: from theory to practice*. 2020;p. 111–166.
63. Baydin AG, Pearlmutter BA, Radul AA, Siskind JM. Automatic differentiation in machine learning: a survey. *J Mach Learn Res*. 2018;18(153):1–43.
64. Dong S, Ni N. A method for representing periodic functions and enforcing exactly periodic boundary conditions with deep neural networks. *J Comp Phys*. 2021;435: 110242.
65. Lu L, Meng X, Mao Z, Karniadakis GE. DeepXDE: a deep learning library for solving differential equations. *SIAM review*. 2021;63(1):208–28.
66. De Su L, Jiang ZW, Jiang TS. Numerical solution for a kind of nonlinear telegraph equations using radial basis functions. In: *International Conference on Information Computing and Applications*. Springer; 2013. p. 140–149.
67. Liu C, Zhu L, Belkin M. Loss landscapes and optimization in over-parameterized non-linear systems and neural networks. *Appl Comput Harmon Anal*. 2022;59:85–116.
68. Jagtap AD, Kawaguchi K, Karniadakis GE. Adaptive activation functions accelerate convergence in deep and physics-informed neural networks. *J Comput Phys*. 2020;404: 109136.

69. Sharma P, Evans L, Tindall M, Nithiarasu P. Hyperparameter selection for physics-informed neural networks (PINNs)-Application to discontinuous heat conduction problems. *Numer Heat Transf B: Fundam*; 2023. p. 1–15.
70. Harmening JH, Peitzmann FJ, el Moctar O. Effect of network architecture on physics-informed deep learning of the Reynolds-averaged turbulent flow field around cylinders without training data. *Front Phys*. 2024;12:1385381.
71. Kashefi A, Mukerji T. Physics-informed PointNet: a deep learning solver for steady-state incompressible flows and thermal fields on multiple sets of irregular geometries. *J Comp Phys*. 2022;468: 111510.
72. Guglielmo G, Montessori A, Tucny JM, La Rocca M, Prestininzi P. Can physical information aid the generalization ability of Neural Networks for hydraulic modeling? arXiv preprint [arXiv:2403.08589](https://arxiv.org/abs/2403.08589). 2024;.
73. Deresse AT. Analytical solution of one-dimensional nonlinear conformable fractional telegraph equation by reduced differential transform method. *Adv Math Phys.* 2022;2022(1):7192231.
74. Jang T. A new solution procedure for the nonlinear telegraph equation. *Commun Nonlinear Sci Numer Simul*. 2015;29(1–3):307–26.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH (“Springer Nature”).

Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users (“Users”), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use (“Terms”). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.

These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.

We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.

While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval, sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.

These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.

To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.

Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.

If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com